

第一讲：计算机的信息表示

2017年9月21日 15:19

第一讲：计算机的信息表示

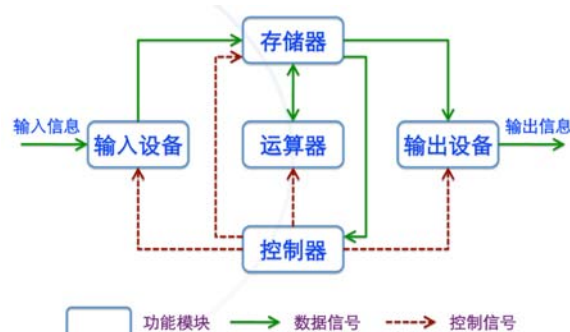
一、计算机的发展历史

一、早期的手工计算辅助工具

- 1) 共同特点：无法记录计算法则、无法设定计算步骤
- 2) 作用：标记计算过程，记录计算结果，进行数字计算的辅助工具

二、现代计算机

- 1) 现代计算机产生于“图灵机”
- 2) 现代计算机的结构：“冯·诺依曼”结构



- 3) 第一台计算机是ENIAC，第一台现代意义上的计算机的EDVAC
- 4) 计算机发展历史：电子管计算机→晶体管计算机→集成电路计算机→大规模集成电路计算机→第五代计算机
- 5) 摩尔定律：集成电路芯片上集成电路的数目，每隔18个月就翻一番
微处理器的性能每隔18个月提高一倍，而价格下降一倍
用一个美元所能买到的计算机性能每隔18个月翻两番
- 6) 计算机分类：微型计算机：PC
小型/中型计算机：服务器
大型计算机：一般的大型公司使用
巨型计算机：国家科技水平的代表
- 7) 1981年，IBM退出了首台个人计算机IBM PC

二、二进制

一、二进制运算

- 1) 数值运算规则：如图所示，跟十进制的算法差别并不大

算术运算规则：

$$\begin{array}{llll} 0+0=0 & 0+1=1 & 1+0=1 & 1+1=10 \end{array}$$

$$\begin{array}{llll} 0*0=0 & 0*1=0 & 1*0=0 & 1*1=1 \end{array}$$

$$\begin{array}{r} 110 \\ + 011 \\ \hline 1001 \end{array} \quad \begin{array}{r} 011 \\ + 011 \\ \hline 110 \end{array}$$

- 2) 逻辑运算规则：与、或、非跟常识相同
异或：两个值相同则异或结果为1，反之则为0

二、进制转换

1) 十进制小数化二进制小数

$$\begin{aligned}0.745 \times 2 &= 1.490 \\0.490 \times 2 &= 0.980 \\0.980 \times 2 &= 1.960 \\0.960 \times 2 &= 1.920 \\0.920 \times 2 &= 1.840 \\0.840 \times 2 &= 1.680\end{aligned}$$

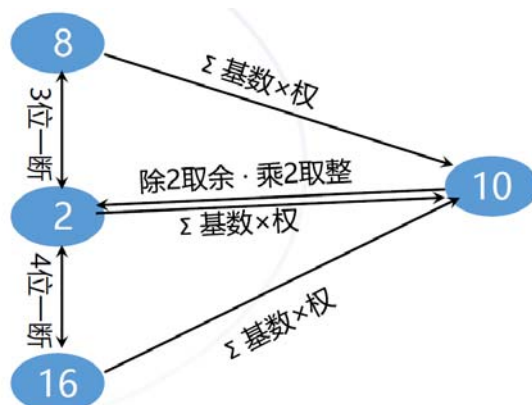
小数部分的转换

乘2取整

在小数部分不为零的情况下，重复的次数越多，结果就越精确

$$\begin{aligned}&(0.745)_{10} \\&\quad \Downarrow \\&(\mathbf{0.101111})_2\end{aligned}$$

2) 其他的进制互换



三、计算机信息的编码

一、二进制信息存储容量的量化单位

1) 位：bit/b/位/比特

最小的信息单位，用0/1表示的一个二进制数位

2) 字节：Byte/B

由8位2进制数位构成

数据存储中最常用的基本单位

一个字节有256个值，可存放一个半角应为字符（ASCII码），用2/4个字节存放一个

汉字

一、数值

1) 常用16位二进制数来表示：无符号数表示范围： $0 \sim 2^{16}-1$

有符号数： $-2^{15} \sim 2^{15}-1$

2) 原码：X=+100 0101 [X]原=0100 0101

X=-100 0101 [X]原=1100 0101

原码的0表示不唯一，不方便进行加减

可用于非数值计算领域

3) 反码：正数的反码和原码相同

负数的反码符号位和原码相同，其余各位取反

0表示仍然不唯一

4) 补码：正数的补码和原码相同

负数的补码符号位和原码相同，其余各位取反，在最末尾+1

0的表法唯一

运算法则： $[X+Y]=[X]+[Y]$; $[X-Y]=[X]+[-Y]$

二、音频

- 1) 基本原理：在时间和波形高度两个维度进行离散化采样
- 2) 每秒钟对音频采用的次数以Hz为单位
- 3) MIDI：另一种声音编码方式

一种电子乐器与电脑之间的协议

类似于矢量图，记录何种乐器，什么音调开始，什么音调结束。

三、图形图像

- 1) 颜色编码及其表示：黑白图像用0、1。灰度图像用二进制编码表示灰度
彩色图像用8位2进制码表示RGB强度
- 2) 图像存储：位图图像和矢量图形

四、字符

1) ASCII码

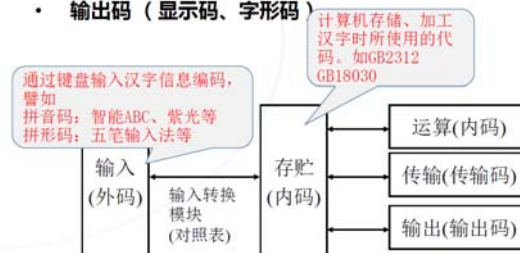
- ASCII (American Standard Code for Information Interchange) 编码规则
 - 每个字符用7位二进制数来表示，7位二进制共有128种状态
 - 最高位为奇偶校验位，或是为0
- 扩展ASCII码 -
 - 8位二进制表示，256个不同的字符
 - 不属于ASCII标准，厂商制定。也有ISO标准

2) Unicode

- 编码方式
 - 使用16位的编码空间。也就是每个字符占用2个字节。这样理论上共最多可以表示 $65,536(2^{16})$ 个字符。
- 实现方式
 - UTF-8: 1-4个字节, 可变长编码, 兼容 ASCII
 - UTF-16: 2或4个字节, Windows NT, Java 采用

3) 中文

- 字符的输入和输出
 - 机外码 (输入码)
 - 机内码 (内码)
 - 输出码 (显示码、字形码)



字形码又有点阵字形描述法和矢量字形描述法

第二讲：计算机的组成与信息的表示处理

2017年11月15日 19:15

第二讲：计算机的组成与信息的表示处理

一、计算机的组成

一、计算机硬件

1. 硬件系统概述

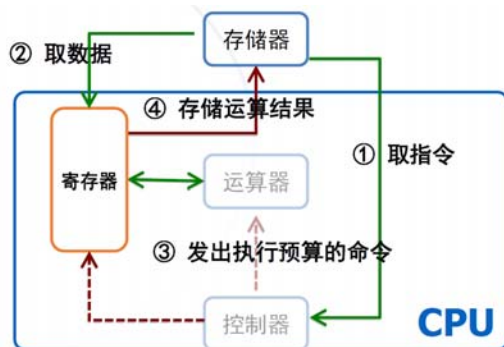
1) CPU：包括了冯诺依曼结构中的运算器和控制器

2) 内部存储器（主存储器）：存储地址：内部存储器以字节为存储单元，每个存储单元都有其特定的唯一的地址

程序 = <指令+>，指令 = <存储码+地址码>

属于随机存储器，具有易失性

3) CPU与内存

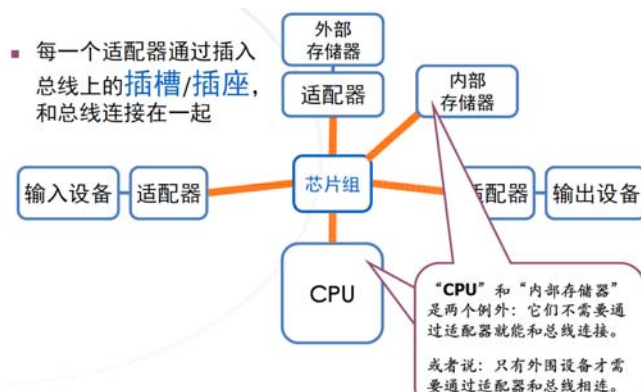


4) 外部存储器：具体例子：硬盘、光盘、软盘等

比较：内存：速度快、容量小、价格贵

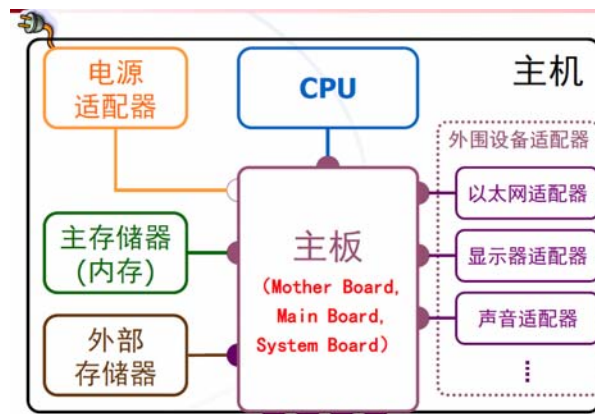
外存：速度慢、容量大、价格低

5) 总线



2. 主机箱内的部件

1) 主机内的硬件设备概览



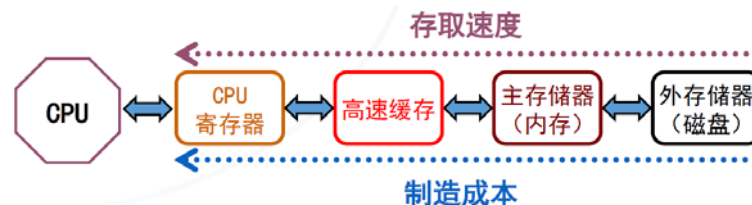
1) 主板-芯片组：集中了主板上几乎所有的控制功能，把以前复杂的控制电路和元件最大限度地集成在几个芯片

内，是构成主板电路的核心。

基本输入输出系统（BIOS）：操作系统最底层部分的可执行程序代码，存放在只读存储芯片里

2) 中央处理器（CPU）：计算机系统的核心设备，其基本功能就是按照程序执行指令
性能主要由主频、字长、地址长度、指令集决定

3) 不同级别的存储设备：主要用来存放计算机指令以及需要存储的数据



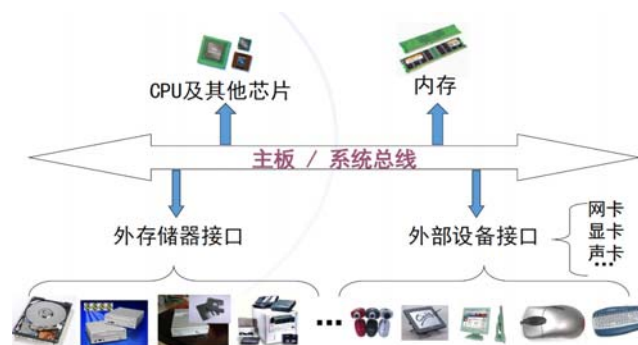
4) 外设插接端口：电源插口、显示器插口等

5) 基本输入输出设备：键盘、鼠标、显示器等

6) 通信设备：网络适配器、无线网卡、调制解调器等

7) 外围设备：打印机、绘图仪、声卡/麦克/音箱等

8) 小结

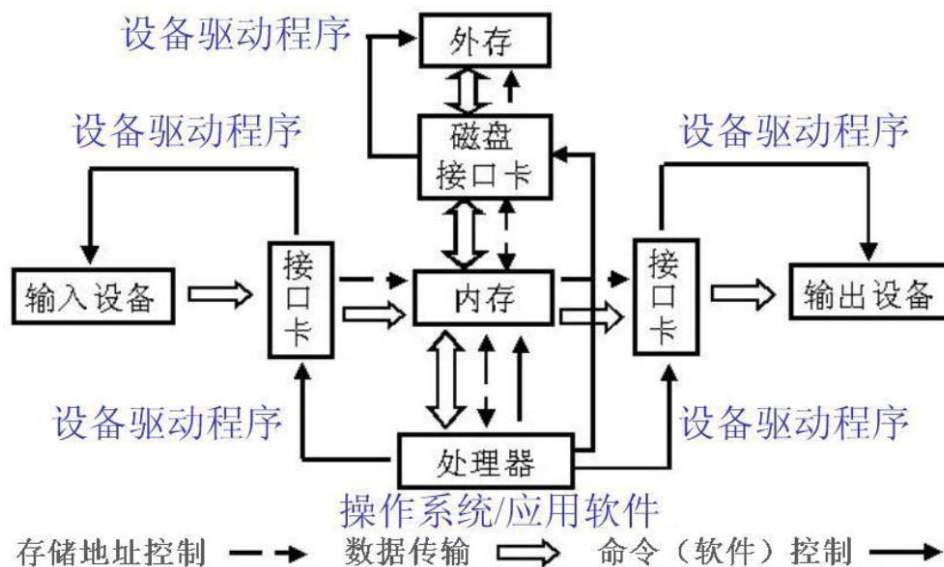


二、计算机信息的存储与处理

一、信息的输入与输出

1.信息的输入

1) 输入的硬件过程



2) 输入的信息类别：计算机程序

数据：计算机程序处理的对象，如文字、数值、图形图像、声音等

操作命令：与计算机系统的交互

用户相应：与应用程序的交互

3) 常用的输入设备：键盘、指点设备、扫描仪、手写板、麦克风、摄像头、数字化仪

2.信息的输出

1) 工作原理：利用自身的物理电路特性，将二进制信息变换为人们易于接受和理解的视听形式（文本、图形、声音）

2) 输出的硬件的过程：见上图

3) 常用输出设备：显示器、投影仪、打印机、绘图仪、音箱

二、信息的存储与管理

1.计算机信息的分层存储原理

1) CPU中的“寄存器”：高速存储单元，其工作速度与运算处理的部件合拍

2) 主存储器（内存）：计算机工作时主存里存放着与当前工作有关的程序数据

3) 高速缓存：为了缓和CPU和主存之间速度矛盾，在CPU和主存储器制建设的一个缓冲性的高速存储部件

4) 外部存储设备：存储方式的一个重要特点就是非易失性，访问速度比主存储器慢得多

5) 硬盘的逻辑分区：实际上是把一个“物理的”是在的硬盘分成几个存储部分

2.信息的管理——文件系统

1) 被存储的除了文件本身的信息内容外，还包括：文件名、类型、位置、大小等其他星系

2) 文件的分类：文本文件：字符

二进制文件：应用程序、图形图像文件、声音文件等

与应用软件相匹配的各种类型的数据文件

3) 文件系统：包括计算机的文件结构和文件组织以及负责管理文件的软件系统，是操作系统的一部分

可以解决的问题：有效的利用外存储器硬件的存储能力，使用多种外存储设备的硬件不同工作方式和特

点

为文件的管理以及在文件上的各种各样的操作提供有效的支

持

功能：文件的创建、命名、更改和删除；文件的类型、属性等的管理；目录（文件夹）的维护管理；文

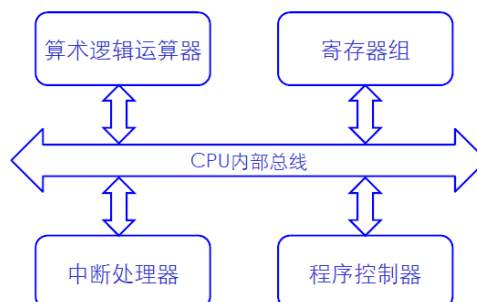
件在磁盘中的存储空间管理；在应用程序中对文件的操作支持

4) 文件的组织结构：目录或文件夹的分层树状结构

5) 数据库管理系统（DBMS）：Oracle/SOL Sever/Infomix/Excel等

3.信息的处理

1) CPU



①CPU的组成-寄存器组：有多个高速寄存器组成的高速存储单元，用于暂时存储运算数据或其他类型的信息

-控制部件：解释指令、根据指令的解释去发出命令；控制计算机其它各部分的活动；对CPU的工作进度和

工作方式的控制

-算术逻辑运算部件ALU：从程序控制部件得到运算指令，再将计算结果存放到寄存器或内存中

-中断处理器：用于处理临时出现的紧急事件，如鼠标移动

②CPU主要性能指标：工作主频：CPU内核工作的时钟频率1.7GHZ~3.0GHZ

运算字长：CPU一次能够处理的二进制位数，32bit/64bit

运算速度：每秒钟执行的指令数，如1000MIPS

③CPU工作周期：读取指令、读取数据、ALU计算，处理中断

2) 主存储器（内存）：程序中的变量和存储单元相对应

主存储器采用随机访问技术

3) 总线

①分组：数据总线：用于传递数据

地址总线：用于传递主存储器地址

控制总线：用于各种控制信息的传递，如读、写等

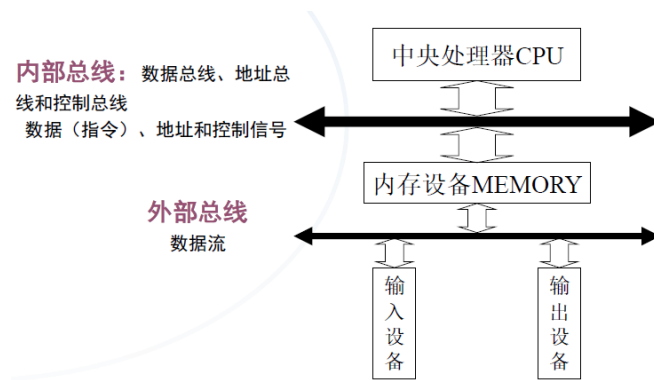
②总线的宽度：数据总线的宽度一般和CPU的字长相一致

CPU地址总线宽度决定了主存储器地址空间的大小：16位地址-64K

32位地址-4G

64位地址-天文数字

③内部总线和外部总线的划分



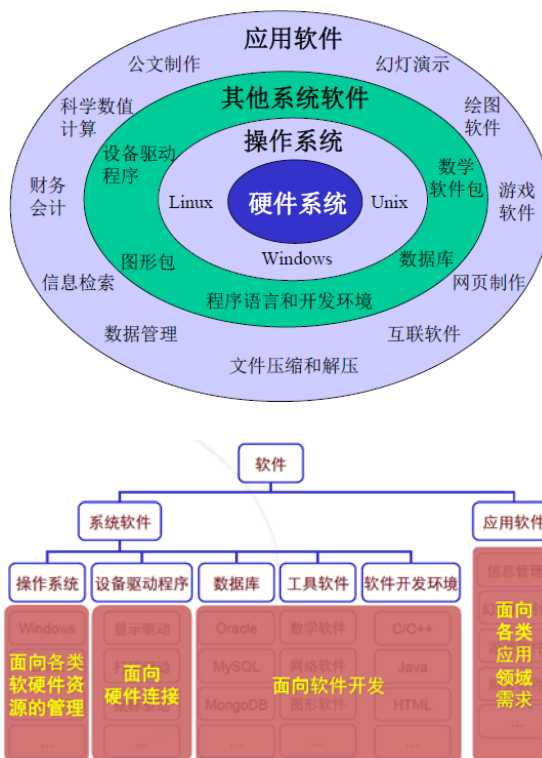
第三讲：操作系统与计算机网络

2017年11月15日 19:16

第三讲：操作系统与计算机网络

一、操作系统

1. 计算机软件系统的层次关系



2. 操作系统的概念

操作系统软件的主要任务是管理计算机系统的硬件资源和信息资源（程序和数据）。此外它还要为计算机上各种硬软件的运行及其相互通信提供支持，并为计算机的用户和管理人员提供各种服务

3. 操作系统的引导

在计算机的电源接通，硬件开始工作后，首先必须把操作系统的**常驻内核**从磁盘装入主存储器，并且使它进入正常工作状态，这样的一个过程成为操作系统的引导，这一过程通过BIOS完成。

4. 操作系统的功能

- 1) 计算机各种硬件资源的管理。主要包括CPU的调度与管理，主存储器及虚拟存储空间的分配和管理，输入输出设备管理及其通信支持等
- 2) 对磁盘存储的信息资源的管理。其功能主要是实现计算机的“文件系统”
- 3) 保证计算机系统的安全性以及计算机系统对它所执行的当前各项任务的监控等任务

二、互联网简介

1. 计算机网络

1) 计算机网络是一种将处于不同地理位置且具有独立功能的多个计算机系统通过通信设备和线路链接起来，在功能完善的网络软件的支持下，实现彼此之间的数据通信和资源共享的系统

2) 基本组成：计算机系统、同轴线路和通信设备、网络协议、网络软件

2. 互联网

1) 互联网的分类：关于广域网、局域网、内联网

2) 局域网中的硬件成分：计算机-网卡

连接线路-有线：双绞线；无线

网络设备：集线器、交换机、路由器

3) 网卡：在网卡存储器中保存一个全球唯一的网络节点地址，用12个六进制数表示，长度为48bit

4) 集线器：星型结构的网络中每个节点都有一条点到点链路与中心节点（交换机、集线器等）相连

5) 交换机：交换机为每个端口提供专用的带宽，每个站点有一条专用链路连到交换机的一个端口，这样

每个站点都可以独享通道，独享带宽。

6) 路由器：路由器用于连接不同的网段并且找到网络中数据传输最合适的路径

路由器主要克服了交换机不能路由转发数据包的不足

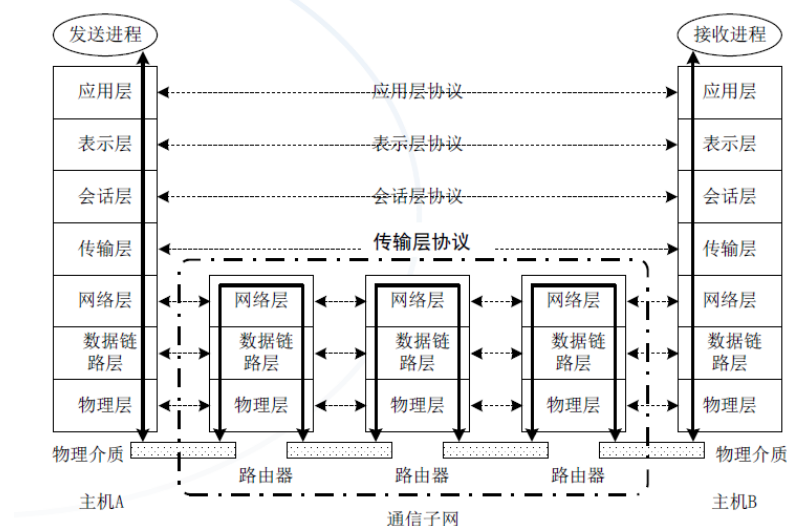
7) 无线网技术：蓝牙技术、红外无线通信技术、Wifi无线局域网、GPRS技术和TD-SCDMA技术

3. 网络协议

1) 协议的三个要素：语法、语义、时序

2) 协议分层：彼此相邻的两层间下层为上层提供通信能力或操作为屏蔽其细节的过程，上层可看成是下层的用户，下层是上层的服务提供者

协议分层—OSI参考模型



3) IP协议：负责因特网地址的使用并规定网络路由器设备所完成的工作，规定了网络层设备如何正确地路由

由并传递IP数据包

IPv4：采用32位2进制数（4字节）表示IP地址

IPv6：采用128位2进制数（16字节）表示IP地址

4) DNS-域名系统：一个将域名映射成相应IP地址的服务系统



5) TCP协议：UDP协议是一种无连接的面向消息的协议，TCP协议是一种有连接的面向流的协议。它是一个界定如何把信息分包的协议（把数据流分割成适当长度的报文段），并且通过给IP包编号保证数据的顺序。

6) 网络协议：集线器工作于物理层，交换机工作于数据链路层，路由器工作于网络层

三、互联网应用

1.基本分类

1) 初期主要是文字类的信息服务

电子邮件、文件传输、远程终端

2) 多媒体类的信息服务

www、即时通信、视频点播、P2P下载

2.互联网应用的基本模式

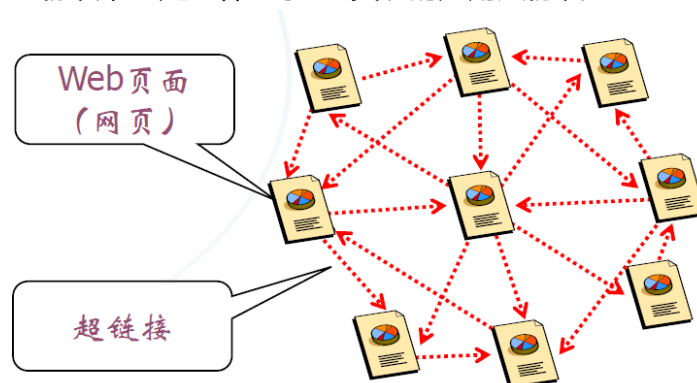
1) 客户端-服务器模式：客户端主动向服务器发出请求

服务器被动地接受来自客户端的服务请求

2) 客户端和服务端仅仅是一种相对的角色，

3.三种经典应用

1) 万维网：使用HTTP协议，它是一种基于TCP实现的应用层协议



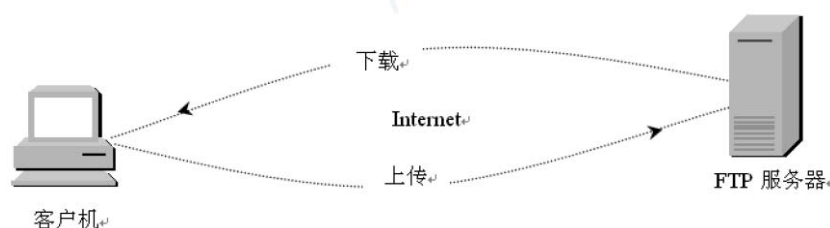
Web页面的源文件采用HTML语言

超链接：超文本链接

2) 电子邮件：电子邮件地址：用户名@邮件服务器域名

发送方和邮件服务器之间采用SMTP协议，服务器和邮件接收方采用POP3协议

3) 文件传输：文件传输协议FTP



第四讲：计算机指令与程序

2017年11月15日 19:16

第四讲：计算机指令与程序

一、指令系统

1.指令用来规定CPU所能完成的基本功能

1) 每一条指令用若干字节的二进制编码表示，包括它要完成的动作及其连带参数（指令系统）。

2) 指令系统中的指令都是计算机能够懂得识别的指令

2.程序是由若干条指令的顺序排列组成，是为了信息处理任务而预先编制的工作执行方案

1) 程序是CPU最频繁使用的信息，应当放在内存中

2) CPU开始执行程序的时候，必须知道程序在内存中存放的位置信息

3) 程序在内存中顺序存储，指令存储的顺序决定了它们在被执行过程中的基本顺序关系（时间关系）

3.汇编指令的分类

1) 存储访问指令、算术运算指令、逻辑运算指令

2) 条件判断和分支转移指令、输入输出指令

3) 其他用于系统控制的指令

二、程序与算法

1.程序和算法

1) 算法：为某个处理任务而预先编制的指令的组合方法，通常称为算法，用于表示问题的求解步骤

2) 算法特性

通用性：合理的输入产生正确的输出

有效性：有限的指令，可预期的执行结果

确定性：每一步的下一步都是确定的

有穷性：算法的执行应该在有限步内结束

2.三种控制结构

1) 顺序结构：按指令自然顺序执行

2) 分支结构：依判断条件决定一条直行路线

3) 循环结构：由循环条件和循环体两部分组成

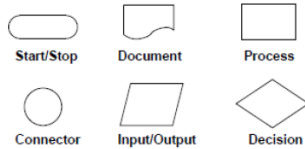
3.常用的算法表示法、

1) 自然语言表示法

2) 伪代码表示法

3) 流程图表示法

- 开始和终止
- 流程线
- 输入输出
- 处理
- 条件判断

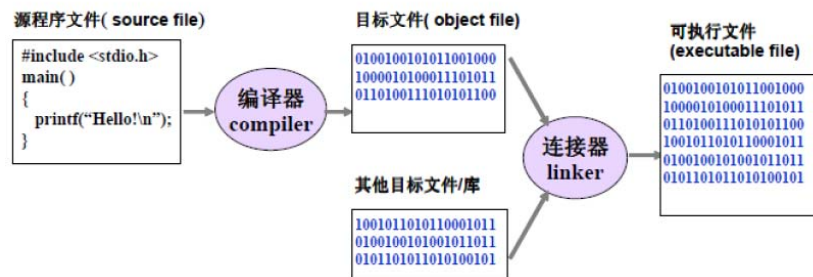


4.基本算法

- 1) 数值算法：加减乘除，最大最小值，解方程，求微积分等
- 2) 非数值算法：排序，查找，文本处理，流程处理等

5.程序设计语言

- 1) 程序设计语言是人们描述（编制）程序所使用的规范和方法，分为机器语言、汇编语言和高级语言
- 2) 程序的翻译：源程序需要被翻译为机器能够识别的机器语言程序，这就是程序的翻译。解释是一次解释并执行源程序的一行；编译是一次将所有源程序法以为目标程序然后执行。
- 3) 编译：将源程序翻译为二进制的目标代码，最后将各个目标代码组装链接为可执行程序



第六讲：C语言程序设计基础

2017年11月15日 19:17

第六讲：C语言程序设计基础

一、简单的C语言源程序

1.C语言

C语言用来编写了UNIX

C语言逐步发展成为开发系统软件的主要语言例如AUTOCAD，Mathematica

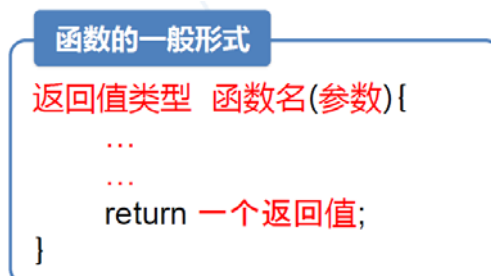
2.C++语言

C++是在C语言基础上发展的一种“面向对象”语言

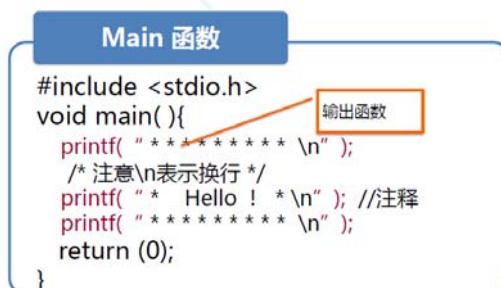
以支持“面向对象”的程序设计方法为基本目标，提供了一套支持面向对象程序设计的机制，如“class”，“object”等

3.C语言是一种函数式语言

1) 函数的一般形式：注意图中两个大括号的位置



2) Main函数有且只有一个

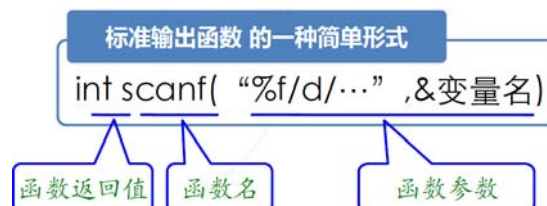


注：\n表示回车换行，\t表示

3) 标准输出函数

4) 标准函数使用规则:在使用C标准库中的任何一个函数之前，都要先包含#include 在该函数所在的头文件。例如

printf (print off) 是stdio.h 中的一个函数



5) 完整的程序实例

求圆的面积

```
#include <stdio.h>      /*预编译：文件包含*/

int main() {
    float s1;            /*定义变量*/
    float r1;

    printf( "请输入第一个圆的半径：" ); /*计算圆的面积*/
    scanf("%f", &r1);          /*输入半径r1的值*/
    s1 = 3.14*r1*r1;          /*求面积*/
    printf( "\n半径为%f的圆的面积为：%f\n" , r1, s1);
    return (1);
}
```

二、变量、常量和数据类型

1.常量

1) C程序中的常量用文字串表示，它区分为不同的类型。常量是具有一定范围的。

- 整型常量：123, -123
- 长整型常量：123L, -123L
- 无符号整型常量：10000U
- 16进制常量：0x12
- 8进制常量：022
- 单精度浮点常量：1.23f, -1.23e12f
- 双精度浮点常量：1.23, -1.23e12

数据类型包括了数据的地址（起始地址）和数据的大小（字节数）

2) 如何表示常量： $x*10^y$ 。其中x是一个浮点常量，y是一个整型变量表示的例子如图：



定义常量：**#define PI 3.14159** 其中PI是别名，3.14159是常量的内容。这一句写在#include那句下面

2.变量

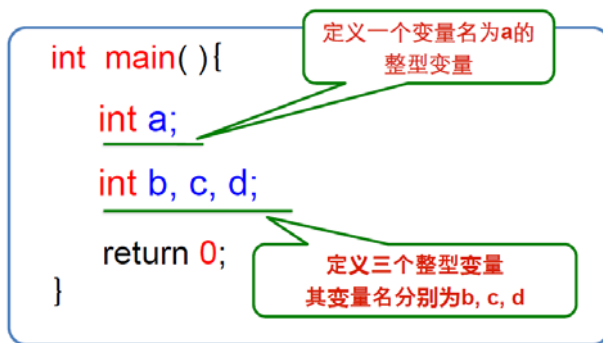
1) 定义变量：数据类型+变量名譬如：**float s1;**

2) 变量是如何实现的：任何一个变量都实现为/对应着内存中的若干连续字节。

3) 变量类型

①整型变量-整数类型的变量，对应关键字是**int**

可以同时定义一个或多个整型变量



可以通过**基本的赋值语句**来给变量赋值

```
int main(){
    int a;
    int b, c, d;
    b = 5;
    c = b;
    d = c - 2;
    a = b*(c + d) - 10;
    return 0;
}
```

通过**标准输出函数**输出整型变量x

将一个变量名为x的整型变量的值输出到控制台的一种方式：

```
printf( "%d" , x);
```

注意在输出整型变量的时候函数参数里写的是**%d**

输出多个变量的时候不让它们输入到一起，还应该加上**\n**

```
printf( "%d\n%d\n%d\n%d\n", a, b, c, d);
```

通过**标准输入函数**输入一个整数值

```
int x;
scanf( "%d" , &x);
```

记住在输入变量的时候要带上**取址符&**

一次输入多个值，在scanf的格式控制中，还可以控制多个输入数据之间的分割标记

- scanf("%c%d%f" , &c, &n, &f) , 则输入数之间以换行（回车）或空格作为分隔;
- scanf("%c,%d,%f" , &c, &n, &f) , 则在输入数据时各数据之间用 “,” 作为分割;

在printf中的格式控制中，还可以包含其他希望输出字符（串），如：

- printf("\nThis is %d!" , n);
- 如果n的值为10，则输出效果为

<空行>
This is 10!

②浮点类型变量：单精度浮点变量对应的关键字是**float**,双精度浮点变量对应的关键字是**double**。前者4字节，后者8字节。一般就用double

输入输出的格式要求基本一样

将一个变量名为x的float型变量的值输出到控制台的一种方式：

```
printf( "%f" , x);
```

将一个变量名为x的double型变量的值输出到控制台的一种方式：

```
printf( "%lf" , x);
```

控制输出的浮点数小数点的位数

```
printf("%.2lf\n", a);
```

```
return 0;
```

保留小数点后2位

在VC输出界面中，浮点数小数部分的缺省输出位数为6位，当然也可以手动控制它保留的小数位数。

③字符型：char

3.数据类型

1) 基本数据类型

■ 基本数据类型 (Primary Data Types)

数据类型说明	数据类型标识符	所占位数	其他
字符类型	char	8位 (1字节)	还可以用unsigned来限定char类型和所有整数类型。unsigned限定的数总是正数或零。
整数类型	int	与机器相关，现在通常为32位(4字节)	
短整数类型	short	int的一半	
长整数类型	long	int的2倍，但现在与int一样，为32位	
单精度浮点类型	float	32位 (4字节)	
双精度浮点类型	double	64位 (8字节)	
无类型	void		函数无返回值

一个数据类型所占的具体字节数，可以通过sizeof运算符来确定。 sizeof(int)

2) 整型数据的分类：基本型：用int表示

短整型：以short int或short表示

长整型：以long int 或long 表示

3) 整型变量的存储格式：4个字节， $-2^{31} \sim 2^{31}-1$

4) 浮点型变量的存储格式：float4个字节，double8个字节，分为符号部分、幂次部分和小数部分

三、标识符和关键字

1.标识符

1) 变量名称的命名规则

①可以由字母、数字或下划线组成

②必须以字母或下划线开头

③不能与C语言的关键字相同

C语言的关键字

int、char、float、double、short、long、
unsigned、struct、union、enum、auto、
extern、register、static、typedef、goto、
return、sizeof、break、continue、if、
else、do、while、for、switch、case、
default、void、entry、include、define、
undef、ifdef、ifndef、endif、line

四、运算符和表达式

1. 表达式

1) 任何一个常量都是一个表达式，任何一个变量都是一个表达式，任何一个合法赋值语句的右边都是一个表达式

2) 表达式的一个特点

表达式的值：一个表达式计算后所得的值，成为表达式的值

表达式的类型：一个表达式的值的类型，称为表达式的类型

3) 常量表达式

一个不带小数点的数字常量，其类型是int

带有小数点的数字常量，其类型是double

末尾字符为f的小数数字常量，其类型是float

`f = 123.4f`

4) 变量表达式

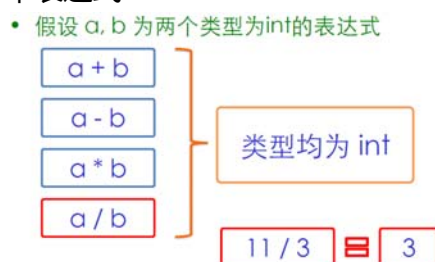
对于一个变量表达式a，它的值就是变量a的值，它的类型就是变量a的类型，赋值的方向会改变变量值的类型

5) 算术运算表达式

①定义：使用算术运算符和括号，按照一定的规则，将**操作数**连接起来的表达式，称为算术运算表达式

②操作数：任何类型为int, float或double的表达式，都可以作为算术运算表达式的操作数

③由**两个整型操作数**构成的算术表达式



注意整型操作数构成的算术表达式结果仍然是整型

另外**取余是%**

④由一个**浮点操作数**和一个**整型或浮点操作数**的算术表达式



在这种表达式里面就没有取余运算

要注意我们在考虑表达式的时候，运算顺序会影响你最后的输出结果。

首先，计算过程在赋值过程之前会产生一定的问题，例如我们要计算分数的浮点，y, x都是整型变量

```
#include <stdio.h>
int main(){
    int x, y;
    double r;
    scanf("%d%d", &x, &y);
    r = y / x;
    printf("%lf\n", r);
    return 0;
}
```



```
#include <stdio.h>
int main(){
    int x, y;
    double r, a, b;
    scanf("%d%d", &x, &y);
    a = x; b = y;
    r = b / a;
    printf("%lf\n", r);
    return 0;
}
```



然后，对于算术运算表达式而言，运算的顺序从左往右，那么运算顺序也会影响最终的结果，还是上面的例子，我们也可以这样写

```
#include <stdio.h>
int main(){
    int x, y;
    double r;
    scanf("%d%d", &x, &y);
    r = 1.0*y / x;
    printf("%lf\n", r);
    return 0;
}
```



进一步更鲜明的例子有：

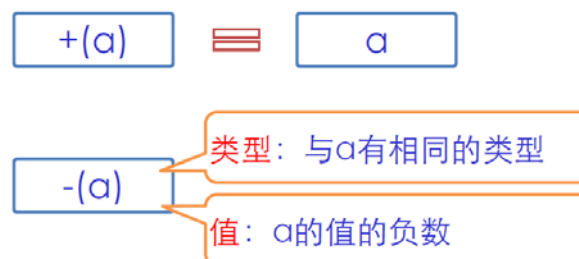
```
#include <stdio.h>
int main(){
    double r;
    r = 1.0*3/2;
    printf("%lf\n", r);
    return 0;
}
```

1.5

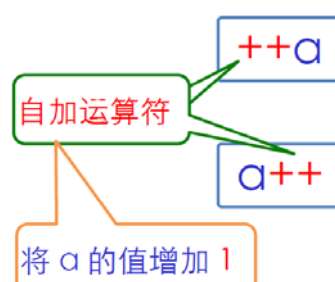
```
#include <stdio.h>
int main(){
    double r;
    r = 3/2*1.0;
    printf("%lf\n", r);
    return 0;
}
```

1.0

⑤由一个整型或浮点操作数构成的算术运算表达式

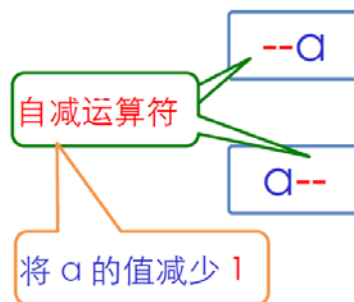


⑥由一个整型或浮点变量构成的算术运算表达式



a++：先取值后运算， $b=a$, $a=a+1$ 。这个表达式的值是 a 的值，类型相同

++a：先运算后取值， $a=a+1$, $b=a$ 。这个表达式的值是 $a+1$ ，类型相同



自减运算符与上面的完全类似

6) 赋值表达式

①存储与类型转换

```
float f = 23; // f 中存储的是23.00000
```

```
int i = 3.56; // i 得到的是3
```

不管=右边的操作数是什么类型，都要转换为=左边变量类型

②数据类型转换

强制类型转换的例子： (int) 就是强制类型转换符号

```
double a = 4.148;  
int b = (int)a;
```

③复合赋值运算符： $+=$ ， $-=$ ， $*=$ ， $/=$ ， $\%=$

```
x = 8.0; y = 2.0;  
x += y * 2;      x = x + (y * 2)  
x -= y / 3;      x = x - (y / 3)  
x *= y + 4;      x = x * (y + 4)  
x /= y - 5;      x = x / (y - 5)  
int a, b;  
a = 5; b = 2;  
a %= b + 1;      a = a % (b + 1)  
return 0;
```

7) 关系运算表达式：int, float, double, char

①关系运算符： $>$ ， $>=$ ， $<$ ， $<=$ ， $==$ ， $!=$

②关系表达式的值只有0和1，类型均为int，关系成立则值为1，反之则为0

8) 逻辑运算表达式：int, float, double

①或运算： $||$

操作数的类型可以是int, float或double

$a1||a2||...||an$ 类型是int，若 $a1...an$ 值都是0，则表达式值为0，反之它的值是1

②与运算： $&\&$

操作数的类型可以是int, float或double

$a1\&\&a2\&\&... \&\&an$ 类型是int，若 $a1...an$ 值都不是0，则表达式值为1，反之它的值是0

③例子：判断某一年是不是闰年

闰年的充分必要条件是:

“年份能被400整除”

或者

“年份能被4整除，但不能被100整除”

写成逻辑表达式就是

$(x \% 400 == 0) \parallel ((x \% 4 == 0) \&\& (x \% 100 != 0))$

注意：逻辑运算表达式的读取顺序也是从左往右

9) 位运算表达式：int

- & (按位与),
- | (按位或),
- ^ (按位异或),
- ~ (按位非)
- 二进制位逻辑运算
 - <<(左位移): 将左侧操作数的二进制数值向左移动若干位 (由右侧的操作数给出), 移出去的位丢弃, 空出的位用0填补。
 - >>(右位移): 将左侧操作数的二进制数值向右移动若干位 (由右侧的操作数给出), 移出去的位丢弃, 空出的位用符号位 (对有符号数) 或0 (对无符号数) 来填补。

例如: $5 \& 3 = (101) \& (011) = (001) = 1$

位移运算的实质 (在不发生溢出时)

- 左移: $x \ll n$, 相当于 $x * 2^n$
- 右移: $x \gg n$, 相当于 $x / 2^n$



对于无符号数，位移运算就按照上面的运算，对于有符号数，符号位不参与移动

五、语句及控制流

六、C语言标准库函数

第七讲：C语言程序设计基础贰

2017年11月15日 19:17

第七讲：C语言程序设计基础贰

1. 字符型变量 (char)

1. 定义一个字符型变量

```
char a;  
char b, c, d;
```

2. 给一个字符型变量赋予一个特定的值

- 1) 字符型常量要在写的内容上面加上单引号，例如 'x'
- 2) 把一个整数赋值给字符型变量，我们写的整数就是字符的ASCII码

```
char b, c, d;  
b = 'x';  
c = b;  
d = 120;
```

字符 'x' 的ASCII编码
一个整型常量

3. 字符型变量的输入输出

1) 输入的格式控制

在用户输入一个字符后，
把用户输入的下一个字符 存放 到 变量 y 中。

```
char x, y;  
scanf( "%c%c" , &x, &y);
```

```
char x, y;  
scanf( "%c %c" , &x, &y);
```

在用户输入一个字符后，忽略用户接下来输入的连续空格字符；
直到用户输入一个非空格字符，才把这个非空格字符存放 到 变量 y 中。

对于第一种写法，我们如果在输入的时候打a_b，那么输出结果就是x=a y=_而不是y=b，为了纠正这个错误，就需要按照第二种写法，在%c和%c之间加上空格

2) 输出的格式控制

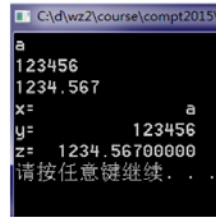
例子：

编写一个程序；

要求：

1. 接收用户输入的一个整数，一个浮点数，一个字符；
2. 将这三个数值分别放在三个变量中；
3. 将这三个变量的值右对齐输出到控制台；
4. 只能调用一次scanf函数；


```
#include <stdio.h>
int main(){
    char x;
    int y;
    double z;
    scanf("%c%d%lf",&x,&y,&z);
    printf("c=%15c\n",x);
    printf("c=%15d\n",y);
    printf("c=%15.8lf\n",z);
    return 0;
}
```



```
C:\d\wz2\course\compt2015\
a
123456
1234.567
x= a
y= 123456
z= 1234.56700000
请按任意键继续...
```

这里就规定了输出的每个变量占十五个字符

2.C程序中的三种控制结构

1.顺序结构

注意先定义再使用

2.分支结构

1.单分支结构：如果表达式的值为真，则执行分支1中的语句，否则，跳过分支1中的语句

例如：取绝对值

```
#include <stdio.h>
int main(){
    int a;
    scanf("%d", &a);
    if(a < 0){
        a = -a;
    }
    printf("%d\n", a);
    return 0;
}
```

接
输

2.双分支结构：如果表达式的值为真，则执行分支1中的语句，否则执行分支2中的语句

```
if(表达式){
    ... //分支1
}
else{
    ... //分支2
}
```

3.多分支结构：如果表达式1的值为真，则执行分支1 中的语句；否则如果表达式2的值为真，则执行分支2 中的语句.....

```

if(表达式1){
    ... //分支1
}
else if (表达式2){
    ... //分支2
}... ..
... ..
... ..
else if(表达式n){
    ... //分支n
}
else{
    ... //分支n+1
}

```

例3：控制台计算器

- 问题描述
 - 通过控制台，实现一个关于两个整数的四则运算计算器
- 程序输入
 - 按照顺序输入：一个整数，一个字符，一个整数
 - 限制1：字符和前后的数字之间用空格间隔开
 - 限制2：字符只能是 +, -, *, / 中的一个
- 程序输出
 - 输入的两个整数的四则运算结果

```

#include <stdio.h>
#include <stdlib.h>

int main( )
{
    int x,y,z;
    char c ;
    scanf("%d %c %d",&x,&c,&y);
    switch(c){
        case '+':{z=x+y;break;}
        case '-':{z=x-y;break;}
        case '*':{z=x*y;break;}
        case '/':{z=x/y;break;}
        default:break;
    }
    printf("x%cy=%d\n",c,z);
    system("pause");
    return 0;
}

```

这里用到的是另外一种多分支语句switch语句，格式如图：

```

switch (expression)
{
    case value1 :
    {
        statements1;
        break;
    }
    .....
    case valueN :
    {
        statementsN;
        break;
    }
    default :
    {
        defaultStatements;
    }
}

```

expression的值类型必须是整型或者字符型

case子句的值value也必须是整型或者字符型常量，而且所有case子句中的值应该是不同的

3.两个函数

1.平方根函数示例

```

#include <stdio.h>
#include <math.h>
int main() {
    double d;
    double r;
    scanf("%lf", &d);
    r = sqrt(d);
    printf("%lf\n", r);
    return 0;
}

```

特别要注意include<math.h>

2.x^y函数示例

```

#include <stdio.h>
#include <math.h>
int main() {
    double x, y;
    double r;
    scanf("%lf %lf", &x, &y);
    r = pow(x, y);
    printf("%lf\n", r);
    return 0;
}

```

4.循环结构

1.while循环的格式

```

while(表达式){
    ... //循环体
}

```

例子：

```

while(i <= 10){
    scanf("%d", &num);

    result += num;

    i++;
}

```

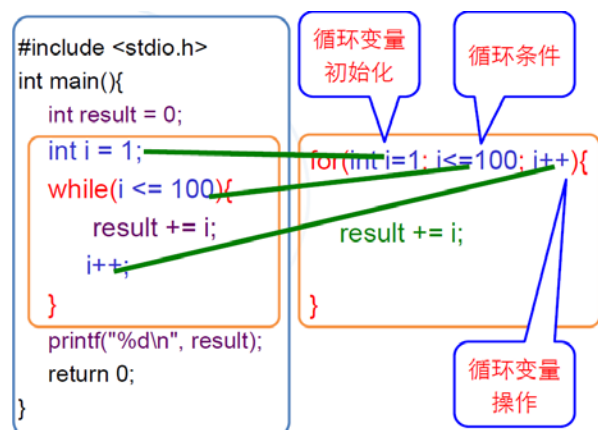
考虑的三个位置：循环多少次，循环控制条件写在哪，变量初始化写在哪。

2.for循环的格式：

- 1) for循环是循环结构的一种简洁形式
- 2) 任何while循环都可以改写为for循环
- 3) for循环与while循环的对应关系



4) 一个例子



3.循环结构中的break;语句（向下跳）

- 1) 功能：跳转到所在循环结构的下一条语句。亦即：跳出循环体。
- 2) 例子：求最大公约数，注意这里while和if的搭配用法

```

#include <stdio.h>
int main(){
    int a, b, result;
    scanf("%d %d", &a, &b);
    result = a;
    while(1){
        if((a%result == 0)&&(b%result == 0)){
            break;
        }
        result--;
    }
    printf("%d\\n", result);
    return 0;
}

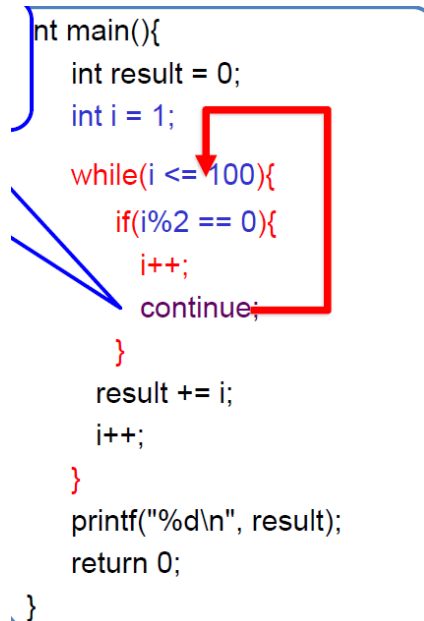
```

4.循环语句中的continue;语句（向上跳）

1) 功能：跳转到所在循环结构的条件判断语句，而不执行这个循环体内接下来的语句。

2) 例子：求奇数的和

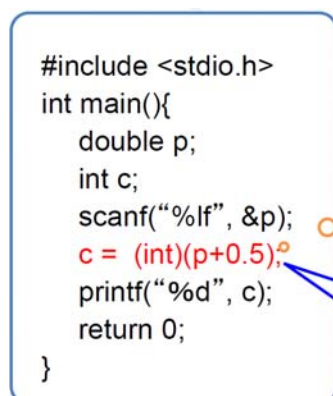
```
int main(){
    int result = 0;
    int i = 1;
    while(i <= 100){
        if(i%2 == 0){
            i++;
            continue;
        }
        result += i;
        i++;
    }
    printf("%d\n", result);
    return 0;
}
```



5.强制数据类型转换

例子：

```
#include <stdio.h>
int main(){
    double p;
    int c;
    scanf("%lf", &p);
    c = (int)(p+0.5);
    printf("%d", c);
    return 0;
}
```



如图的写法是把一个double类型的数四舍五入到一个整数的方法

6.条件表达式

int a = 表达式1 ? 表达式2 : 表达式3;



```
int a;
if(表达式1){
    a = 表达式2;
} else{
    a = 表达式3;
}
```



作为分支结构的一种简单写法

第八讲：数组

2017年11月15日 19:18

第八讲：数组

一、什么是数组

1.数组是一种复合数据类型，从整体上定义了一组类型相同的变量，数组的元素顺序地存储在连续的内存空间中

二、如何定义数组

1.格式：

```
数据类型 变量名[ 数组元素个数 ] =  
    { 数组元素初值 };
```

其中数组元素个数必须是整数常量，其中“={数组元素初值}”部分可以省略

2.注意：第一个数组元素的下标是0，最后一个数组元素的下标是N-1

三、数组元素的赋值与访问

1.数组中的每个变量都可以通过“数组变量名[下标]”来访问

2.一次性赋值：使用for循环，例如：

```
for (i=0; i<10; i++)  
    scanf("%d", &student[i]);
```

3.访问数组元素不要越界！

四、数组的用途

第九讲：字符串

2017年11月15日 19:18

第九讲：字符串

一、前情回顾

编码值为0的字符是什么字符？

编码值为0的字符是NUL，表示一个空值。它的输出效果就是一个空格。但是它跟“空格”这个字符并不是一个字符

a=0等价于a='\0'

二、字符数组

1.与数值型数组相似，字符数组用于一次声明多个字符型变量，例如：

```
char zifu[10];
```

三、字符串

1.字符串是由一个或多个字符构成的序列，存储在一个字符数组里面，由\0结尾

2.如何声明一个字符串常量：用双引号把需要表达的字符串括起来

3.如何声明一个字符串变量：声明一个LEN+1的一维字符数组，例如：

```
char st[20]="you are a student!"
```

4.字符串的长度：字符串中第一个\0字符之前的字符的个数，注意需要#include<string.h>例如：

```
int m = strlen("you are a student!");  
int n = strlen("you are a stud\0ent!");
```

需要 #include <string.h>

m=18; n=14

```
char str[10]={'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j'};
```

只是一个字符数组！并不能表示字符串“abcdefghij”

```
char str[10]={'a', 'b', 'c', 'd', 'e', '\0', 'g', 'h', 'i', 'j'};
```

可以表示字符串“abcde”，\0之后的字符并不关心！

```
char str[10]={'a', 'b', 'c', 'd', 'e', '\0', 'g', 'h', '\0', 'j'};
```

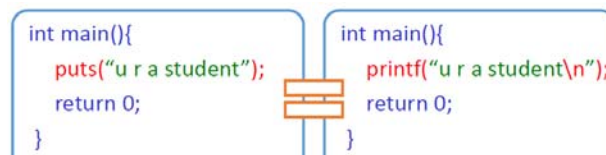
还是表示字符串“abcde”，以第一个\0为字符串的结束标记，即便后面还有\0！

5.向控制台输出字符串常量：

1) 利用printf函数：在这个例子中，就只会输出到\0之前的内容

```
int main(){  
    printf("you are a stud\0ent");  
    printf("\n");  
    return 0;  
}
```

2) 利用puts函数，



6.向控制台输出字符串变量

1) 利用printf函数+%s修饰符

```
int main(){
    char zfc[50];
    ... .. //给字符串变量zfc赋值
    printf("%s", zfc);
    return 0;
}
```

2) 利用puts函数

```
int main(){
    char zfc[50];
    ... .. //赋值
    puts(zfc);
    return 0;
}
```

```
int main(){
    char zfc[50];
    ... .. //赋值
    printf("%s\n", zfc);
    return 0;
}
```

7.字符串变量的赋值

1) 声明时赋值的简洁方式

```
char zfc[LEN] = "u r a student";
```

```
char zfc[LEN];
zfc = "u r a student";
```

2) 声明后赋值方法1：scanf+%s，参数就是数组名，不加取地址符，这种方法不能接收包含空格的字符串，如果有空格它只识别到第一个空格之前为止。

3) 声明后赋值方法2：gets函数，可以接收包含空格的字符串。尽量使用方法1，除非字符串中含有空格

```
char zfc[50];
gets(zfc);
```

小技巧，使用getchar()可以单独读入一个换行

8.字符串的初始化及输入输出小结

初始化

- char str[12] = "I am happy";
- char name[12];

从键盘输入字符串

- scanf("%s" , name); //以空格或回车为结束标记
- gets(name); //以回车为结束标记，可以有空格

输出字符串

- printf("%s" , name);
- printf("%s" , str);
- puts(name); //相当于 printf("%s\n" , name);

9.字符串的常用函数 (需要#include<string.h>)

1) 字符串复制函数strcpy(目标字符串，源字符串)

```
char zfc_src[50], zfc_tgt[50];
gets(zfc_src);
strcpy(zfc_tgt, zfc_src);
printf("%s\n", zfc_tgt);
```

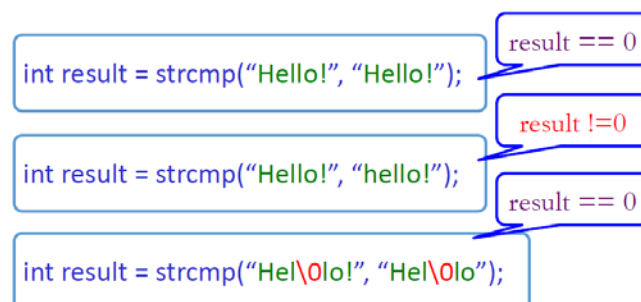
2) 获得字符串长度函数 `int strlen(字符串)`

```
char zfc[100];
gets(zfc);
printf("%d\n", strlen(zfc));
return 0;
```

3) 字符串比较函数：`int strcmp(字符串A, 字符串B)`

当字符串A和字符串B相等时，返回0；

否则，返回两个字符串中第一对不相等的字符对应的ASCII码值的差；举例如下



实际上，我们可以认为这个函数的返回值意义是

返回一个整数，其意义：

0： 字符串1 和 字符串2 的内容相同

正整数： 字符串1 > 字符串2

负整数： 字符串1 < 字符串2

由于我们不能用==来比较字符串的内容，所以可以用这个函数来判断两个字符串是否相等

4) 字符串连接函数 `strcat(目标字符串, 源字符串)`

功能：把源字符串添加到目标字符串的结尾

注意：调用时，要确保目标字符串所在数组的长度够大

5) 大写字母变小写 `strlwr(字符串)` 函数和小写字母变大写 `strupr(字符串)` 函数（这两个函数在编程网格中不支持）

四、字符串与数值

1. 把字符串和变量合并：用函数 `sprintf(字符数组, 格式控制, 变量列表)`

```
char s1[50] = "An int";
int a = 20;
double b = 3.1416;
sprintf(s1, "An int %d and a double %f\n", a, b);
printf("%s", s1);
return 0;
```

2. 只使用一种输入函数，既要输入有空格的字符串，又要输入数值：

1) 另一种常用的字符串函数：用函数 `sscanf(字符数组, 格式控制, 变量列表)`

例：char s1[50] = "88 9";

```
int x, y;
printf("%s\n", s1);
sscanf(s1, "%d %d", &x, &y);
printf("%s", s1);
```

2) `get` 和 `scanf` 不要混用

3) 字符串与数值相互转换的函数：

在输入数值时也使用gets()，得到表示数值的字符串，再将该字符串转换成数值。

int atoi(char *str) : 将字符串转换成整数

double atof(char *str) : 将字符串转换成浮点数

#include <stdlib.h>

例子：

```
char s[100];
double x;
int i;
gets(s); /* Test of atof */
x = atof( s );
printf( "atof test: ASCII string: %s float: %lf\n", s, x );
gets(s); /* Test of atoi */
i = atoi( s );
printf( "atoi test: ASCII string: %s integer: %d\n", s, i );
return 0;
```



五、字符串数组

1.声明字符串数组

第一步：

确定 数组中存储的字符串 的 最大长度LEN

第二步：

确定 数组包含的字符串变量 的 最大数量N

第三步：

声明 行数为N， 列数为LEN+1 的 二维字符数组

char zfc[N][LEN+1];

2.使用字符串数组

```
int main(){
    char zfc[2][50];
    gets(zfc[0]);
    strcpy(zfc[1], zfc[0]);
    printf("%s\n", zfc[1]);
    return 0;
```

第十讲：指针与动态数组

2017年11月16日 15:00

一、指针的概念、定义和使用

- 1.通过**变量的地址**能找到该变量在内存中的**存储单元**，从而访问它的值
- 2.存放地址的变量叫作**指针变量**。例如变量pa是a的指针变量，存储的是a的地址值例如2000
- 3.定义指针变量

```
int a, j, k;
a=100;
j=6;
//scanf("%d", &a);
int *pa;
pa = &a;
*pa =200;
.....
```

pa 这个变量是指针变量，它是a的地址。*pa表示的是pa指向的值，也就是a的值，我们给*pa赋值就是直接改变变量a的值。我们当然也可以写 pa=&b, 将这个指针变量指向b，然后再写*pa=200来直接修改b变量的值。

pa=&b这种步骤叫作“搭桥”，也就是把指针变量和已经存在的某个变量联系起来

其中&是取地址运算符，*是指针运算符

4.指针变量的使用

- 1) 不管指针变量指向数据类型是什么，指针变量在内存中始终占用4个字节或8个字节（地址总线的宽度）
- 2) 指针变量指向数据的首地址，即该数据的首字节地址。（指针就是一个内存存储单元的地址）

```
int a, j, k;
int *m;
a=200;j=6;
m=&a;
printf("%d\n",m);
printf("%d,%d\n",a,*m);
m++;
//m=&j;
printf("%d,%d\n",a,*m);
```

在这个例子中，为什么最后一行*m输出出来之后是6487616=6487612+4呢？这是因为m++是以4个字节为单位加的

- 3) 指针变量的类型对应它所指向的内存区域的大小
- 4) 在一个指针变量中只能存放同一类型变量的地址
- 5) 所有的指针必须先指定类型才能使用
- 6) 一次定义多个指针变量，注意每个指针变量都要有自己的“*”
- 7) 指向**数组元素**的指针变量

```
int scores[5];
int *pointer;
pointer= &score[1];
```

8) 特殊的指针：空类型指针void *pointer

指针pointer指向的那段内存中存储的数据是A(一串01)，该类型指针的含义是：A的数据类型在定义该指针时还不清楚。在使用A时，需要先做强制类型转换，明确指出A的类型。下面是一个例子：

```
main ( )
{
    int a,b;
    void *pointer;
    pointer=&a;
    scanf("%d,%d",&a,&b);
    scanf("%d", (int *) pointer);
    printf("a=%d,b=%d\n",a,b);
    printf("%d",*(int *)pointer);
}
```

二、使用指针访问数组

- 1.指向数组元素的指针，其定义与一般指针变量相同
- 2.可以通过指针为数组元素赋值

```
int a[10], *m;
m=&a[0];
*m=5;
```

- 3.可以改变指针变量指向的数组元素

```
int a[10];
int *m=&a[0];
m=m+1; //m指向数组a[1]
```

- 4.数组的指针是指数组的起始地址

(当指针指向数组首元素时，指针可等同数组变量使用)

```
int scores [5];
int *pscore;
pscore = scores;
// pscore = &scores[0];
*pscore=10;
```

数组名等价于数组第一个元素的首地址。我们现在明白了scores这个表达表示的是一个地址，那么既然可以由scores[0]这个写法，我们做了pscore=scores这个操作之后，就可以用pscore[0]来访问对应的数组元素也就是说，指向数组的指针变量也可以带下标，例如：

```
int scores [5];
int *pscore;
pscore = scores;
// pscore = &scores[0];
*pscore=10;
pscore[1]=100;
```

指针也可以移动，可以使用pscore++/pscore--，但不能使用乘除运算，例如：

```
int scores [5];
int *pscore;
pscore = scores;
// pscore = &scores[0];
*pscore=10;
pscore++;
*pscore=100;
```

指针的加减法运算不是简单的地址值的加减法，而是考虑了类型大小的地址加减法

```
(pscore+1) 的值是 &scores[1]
*(pscore+1) 等价于 scores[1]
*(pscore+i) 等价于 scores[i]
```

指针变量可以用指针运算的结果赋值

```

pscore = scores;
pscore = pscore+1;
pscore++;
//此时 *pscore 与 scores[2] 等价

```

两个指针变量的相加是无意义的，两个指针变量相减是其间的元素数目

```
(pscore2 - pscore1) == 1; // 值为真
```

两个指针变量可比较大小

```

(pscore2 > pscore1); // 值为真
(pscore2 < pscore1); // 值为假
pscore1 == scores; // 两个指针变量所指的地址相同

```

三、小结

1. 恒等式

```
*(amp;a)==a
```

```
*(scores+i)==scores[i]
```

2. 指针用过一遍之后记得初始化，否则容易越界

四、动态数组（内存的动态分配与释放）

1. 数组变量都有固定的大小（元素数目），处理数据的能力再变成实就已经固定，无法改变，在实际使用中很不方便

2. 通过指针变量和malloc()函数，在程序运行期间动态地申请内存，把分配到的内存地址赋值给指针变量，作为**动态数组**来使用

3. 使用结束后，通过free()函数来释放内存，即把内存归还给系统。

```

int maxInt, i, *pint;
scanf("%d", &maxInt);
pint = (int*) malloc( sizeof(int) * maxInt );
for ( i = 0; i < maxInt; i++ )
{
    scanf("%d", pint+i); //pint+i本身就是地址
    //scanf("%d", &pint[i]); //另一种写法
}
for ( i = 0; i < maxInt; i++ )
    printf("%d\n", pint[i]); //以数组方式访问内存

free(pint);

```

指针变量虽然指向的是一个内存地址，实际上通过它可以访问一个连续的内存区域。

4. 注意

malloc函数的参数为所需申请内存的大小：以字节为单位

malloc函数的返回值是空的，必须通过强制类型转换，才能赋值给特定的指针变量

```
int * pint = (int *) malloc( ... );
```

最好使用"sizeof(类型名)*动态数组长度"形式确定动态数组的大小：

```
int * pint = (int *) malloc( sizeof(int) * 100 );
```

分配的内存不再使用时一定要释放

记得#include<malloc.h>

五、字符串以及字符指针

1. 字符指针的使用

用字符串给字符数组赋初值我们可以这么写

```

void main()
{
    char s[12] = "I am happy";
    printf("%s", s);
}

```

也可以定义一个字符指针变量，让它指向字符串常量

```

void main()
{
    char * str = "I am happy";
    printf("%s", str);
}

```

2. 字符串指针变量的值是可变的

```
void main()
{
    char s[12] = "I am happy";
    char *str;
    str = s;
    *str = 'U' ;
    printf("%s\n", str);
    str = s + 5;
    printf("%s\n", str);
}
```

这一段程序的输出结果就是

U am happy
happy

3. 循环复制字符串

1) 不使用字符串指针的写法

循环复制字符串

```
void main() {
    char s[12] = "I am happy";
    char t[20];
    int i;
    for(i=0; s[i]!='\0';i++)
        t[i] = s[i];
    t[i] = '\0' ; //此时i为?
    printf("%s\n%s", s, t);
}
```

s	I	a	m																
t	I	a	m																

2) 使用字符串指针的写法

使用指针变量完成字符串复制

```
void main() {
    char s[12] = "I am happy";
    char t[20];
    char *ps = s, *pt = t;
    for( *ps!='\0'; ps++,pt++)
        *pt = *ps;
    *pt = '\0';
    printf("%s\n%s", s, t);
}
```

		ps	ps	ps	ps	ps	ps	ps	ps	ps	ps	ps	ps	ps	ps	ps	ps	ps	ps
s	I	a	m																
t	I	a	m																
		pt	pt	pt	pt	pt	pt	pt	pt	pt	pt	pt	pt	pt	pt	pt	pt	pt	pt

4. 字符数组只能对各个元素赋值，不能对数组整体赋值

5. 使用字符指针变量前必须对变量赋值，也就是说：

指针必须要指向一个具体的内容，然后才能进行初始化

第十一讲：结构体

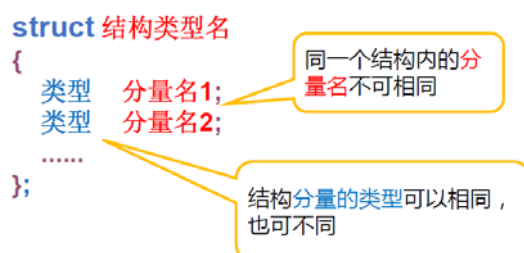
2017年11月23日 15:35

一、为什么要引入“结构”

1. 举例：通常，一个学生的个人信息包括：学号、姓名、性别、年龄、各门功课的成绩等数据。这些数据都与一个学生相关联，但是数据类型各不相同。应该把它们组织成一个组合项，把它们当做一个有机的整体
2. 上面所说的组合项就是结构。结构是一种新的数据类型

二、结构体类型及其定义

1. 定义一个结构类型



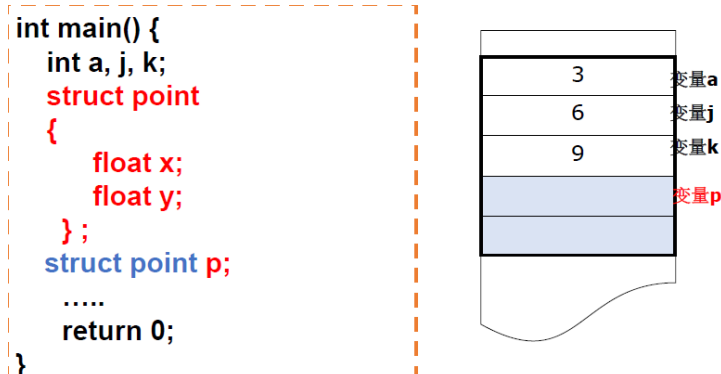
例如我们可以用下面的结构来表示“一个学生的个人信息”

```
struct Student
{
    int number;
    char name[8];
    char sex;
    int age;
    float course[8];
};
```

三、结构变量

1. 定义

定义**新结构**只是定义了一种新的**数据类型**，系统并不为这个**新类型**分配内存空间，而只是为这个类型的**变量**分配空间



也就是说，在上面这个例子中，我们定义的结构类型叫作struct point，这个类型下面的变量p才有空间

2. 结构变量定义的两两种形式

用以定义的结构定义变量/定义结构的同时定义结构类型的变量

```

struct point
{
    float x;
    float y;
};
struct point p1, p2;

```

```

struct point
{
    float x;
    float y;
} p1, p2;

```

3. 结构体的大小并不完全决定于分量（并不是分量大小的简单加总），这是编译器在编译时的一个特殊要求

4. 访问结构体中的分量：通过“变量名.分量名”完成

```

float dx, dy;
struct point
{
    float x, y;
} p1, p2;

p1.x = p1.y = 3.5f;
p2.x = p2.y = 1.5f;
dx = p1.x - p2.x;
dy = p1.y - p2.y;

```

5. 结构变量本身可以作为一个整体来使用，结构变量的值由各个分量构成

6. 结构体数组：可以形象地理解为一个向量组

```

float dx, dy;
struct point
{
    float x, y;
} p1, p2, points[2];

p1.x = p1.y = 3.5f;
p2.x = p2.y = 1.5f;
points[0] = p1;
points[1] = p2;

```

四、各种类型的结构分量

1. 结构分量在内存中顺序存放

2. 结构分量的类型可以是任何类型

3. 分量的类型不能是**未定义的结构类型**或者**正在定义的结构类型**

```

struct city
{
    char name[32];
    struct city newcity;
} x;

```

✗

```

struct city
{
    char name [32];
    struct city *pcity;
} x, y;

```

✓

为什么最后一种定义方法是对的呢？因为系统能够找到位置和大小，后面指针的大小是确定的。相比之下，前面的大小不定，里层大小去决定外层大小，这就构成了矛盾。

五、应用实例

1. 学生成绩统计

首先我们定义了一个有160人的班级的学生信息结构数组

```

struct student {
    int number;
    char name[8];
    char sex;
    int age;
    float course[8];
};
struct student class1[160]

```



例如，如果要统计所有学生course[1]的平均成绩，我们可以通过class[i].course[1]来访问每个学生course[1]的成绩

六、指向结构体变量的指针

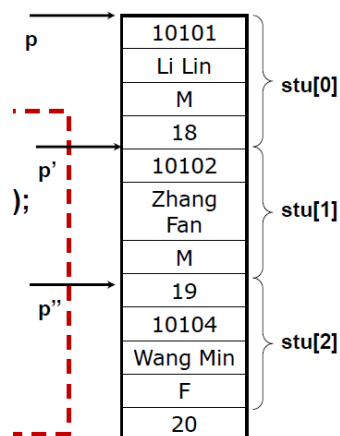
1. 一个结构体变量的指针就是该变量占据的内存段的起始地址
2. 通过“指针结构变量名->成员名”进行访问

```

struct student {
    int num;
    char name[20];
    char sex;
    float score;
};
struct student stu1;
struct student *p;
p=&stu1;
struct student stu[3]={ {10101,"Li Lin",'M',18},
{10102,"Zhang Fan",'M',19}, {10104,"Wang Min",'F',20}};

struct student *p;
printf(" No. Name Sex age\n");
for (p=stu;p<stu+3;p++)
    printf ("%5d %10s %2c\n", p->num,p->name,p->sex,p->age);

```



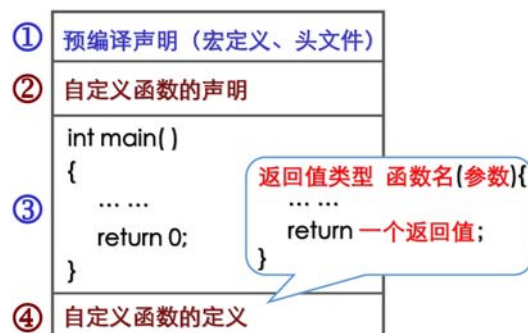
这里面，其实p->num等同于(*p).num

第十二讲：函数、函数调用

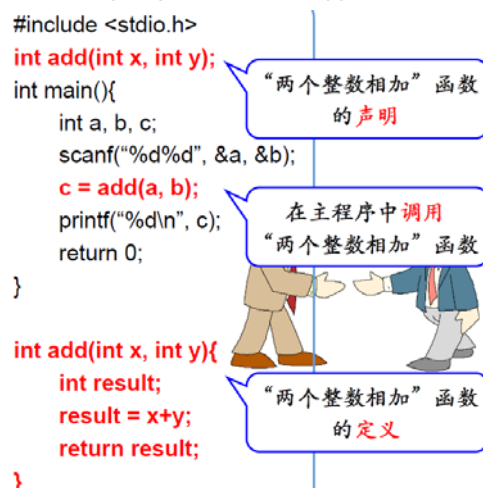
2017年11月30日 15:05

一、函数的声明和定义

1. 自定义函数的总体程序结构



例如：我们把“加法”封装到一个函数中，就可以这样写

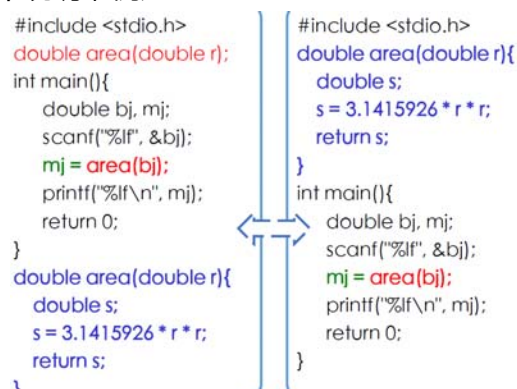


其过程为函数声明→函数调用→函数的返回值赋值给变量→函数定义

二、函数调用

1. 被调用的函数必须具备的条件

- 1) 被调用的函数必须是已经存在的函数
- 2) 若调用库函数或者自定义函数，应该在文件开头用#include命令将所需的信息包含到本文件夹中
- 3) 一般应该在开头或主调函数中对被调函数的类型进行说明，但有个例外：被调函数的定义体写在主调函数之前可以不说明，例如：



2. 实参与形参

1) 形式参数

有参函数在定义时，函数名后面括弧中的变量名称为“形式参数”，简称形参。形参只能是变量，必须指定形参的类型

2) 实际参数

在调用函数时，函数名称后面括弧中的表达式称为“实际参数”，简称实参。实参可以是常量、变量或表达式

实参和形参应该个数一样、类型一致

3) 实参和形参的关系是单向传递，只由实参传递给形参

①在调用的时候，形参的值如果发生改变，并不会改变主调函数的实参的值

例如：我想用一个函数来完成两个变量值的交换，如果这样写是不行的

```
void swap(int x, int y)
{
    int tmp = 0;
    tmp = x;
    x = y;
    y = tmp;
}

void main()
{
    int a=0, b=0;
    a = 20;
    b = 45;
    if(a<b) swap(a, b);
    .....
}
```

因为a,b的值可以赋给x,y但是交换过后的x,y无法反过来赋回给a,b（形参如果没有return就无法把操作过程还给实参）

但是可以用指针去写这个函数

```
void swap(int *x, int *y)
{
    int tmp = 0;
    tmp = *x;
    *x = *y;
    *y = tmp;
}

void main()
{
    int a=0, b=0;
    a = 20;
    b = 45;
    if(a<b) swap(&a, &b);
    .....
}
```

这时候的调用过程是，把a和b的地址给了指针x,y，然后把这两个指针指向的值互换了，从而a和b原来对应的地址上的值就互换了

②如果实参是数组名，则传递的是数组首地址而不是变量的值

```
void func( int b[] ) {
    int j;
    for(j=0;j<4;j++)
        b[j]=j;
}

main ( ) {
    int a[4],i;
    func(a);
    for(i=0;i<4;i++)
        printf("%d ",a[i]);
}
```

运行结果：0 1 2 3

三、函数的返回值

1.函数的返回值

函数执行完以后可以向调用它的程序返回一个值，表明函数的运行状况或运算结果

函数通过return语句的一般形式是：

return 表达式;

表达式可以用括号括起来。返回值可以有不同的类型，返回值类型在函数定义的时候就要说明，例如

```
double calculate( int a, double b);
//返回值类型为double，有一个整型参数，一个双精度浮点型参数
//函数名为calculate

char judge( );
//返回值类型为char，没有参数，函数名为judge

void swap(int *x, int *y);
//返回值类型为void，表示不返回任何值，有2个整型指针参数
//函数名为swap
```

2.一旦函数执行了return语句，则表明函数任务已经完成了，函数到此结束了

3.当想要返回多个返回值的时候，建议使用指针（引用）

例题：编写一个函数选择数组a[10]的最小值和次小值的下标

```
void Select(int a[ ], int n, int *x1, int *x2) {
    int m1=100,m2=100, j;    // m1最小， m2次小
    for(j=0; j<n; j++)
    {
        if(a[j] < m1) /*是否小于最小的？ */
        {
            m2 = m1; *x2 = *x1;
            m1 = a[j]; *x1 = j; /* 改变最小的 */
        }
        else if ( a[j] < m2 ) /*是否小于次小的？ */
        {
            m2 = a[j]; *x2 = j; /* 改变次小的 */
        }
    }
}

#include <stdio.h>

void Select(int a[ ], int n, int *x1, int *x2) ;
int main( )
{
    int a[10], i;
    int minx1, minx2;
    for ( i=0 ; i<10; i++ )
        scanf("%d", &a[i]);
    Select(a,10,&minx1,&minx2); //函数调用
    printf("\n最小值a[%d]=%d ", minx1, a[minx1] );
    printf("\n次小值a[%d]=%d ", minx2, a[minx2] );
    return 0;
}
```

4.当想要返回多个返回值的时候，还可以使用全局变量

- 1) 在程序中，每一个变量都有一个可以起作用的范围，这个范围就是变量的作用域
- 2) 根据变量的作用域的不同，变量可分为**全局变量**和**局部变量**。

```

#include <stdio.h>
int total; //全局变量
int tryit(int a);

int main() {
    int n, j;
    total = 0;
    for( j=0; j<1000; j++ ) {
        scanf("%d", &n);
        printf("%d\n", tryit(n));
    }
    printf("%d\n", total);
    return 0;
}

```

3) 例题：从数组中选择最小和次小的值

```

#include <stdio.h>
void Select(int a[], int n) :
int minx1,minx2;

int main() {
    int a[10], i;
    for ( i=0 ; i<10; i++)
        scanf("%d", &a[i]);
    Select(a,10); //!!!!!!
    printf("\n最小值a[%d]=%d ", minx1, a[minx1] );
    printf("\n次小值a[%d]=%d ", minx2, a[minx2] );
    return 0;
}

void Select(int a[], int n)
{
    int m1=100,m2=100, j; /* m1最小, m2次小 */
    for(j=0; j<n; j++)
    {
        if(a[j] < m1 )
        {
            m2 = m1; minx2 = minx1; /* 最小到次小 */
            m1 = a[j]; minx1 = j; /* 改变新的最小 */
        }
        else if( a[j] < m2 )
        {
            m2 = a[j]; minx2 = j; /* 改变次小的 */
        }
    }
}

```

使用全局变量的时候，我们可以看到这里不需要使用指针了，因为这里主调函数和被调函数使用统一变量，在被调函数中形参改变了，主调函数里面实参也会跟着改变

第十三讲：简单排序与检索算法

2017年12月7日 15:12

0.方法论

1.在C语言编程中，我们要练习自己分解问题的能力，把一个大问题转化成很多“函数”可以解决的小问题，例如假币问题

2.在编写程序之前，一定要有一个“设计”的阶段：

- 1) 要用什么样的类型存储数据？
- 2) 要用什么算法来解决问题？

1.如何评价一个算法

1.衡量算法的优劣

- 1) 时间效率：执行算法耗费的时间；
- 2) 空间效率：运行算法需要耗费的存储空间，其中主要考虑辅助空间
- 3) 理解、阅读、编写、调试的难易等

2.评价算法的空间代价和时间代价

- 1) 事后统计法：必须先运行程序

所得时间统计量依赖于计算机软硬件等环境因素

- 2) 事前估算法的影响因素：算法的策略

问题的规模 (N)

程序语言

编译程序所产生的机器代码的质量

机器执行指令的速度

- 3) 时间复杂性：当问题的规模以某种代为由1增至n时，对应算法所耗费的时间也以某种单位由F(1)增至F(n)，这是我们称该算法的时间代价是F(n)

时间单位：执行一个简单语句（如赋值语句、判断语句等）所用时间

- 4) 空间复杂性：当问题的规模以某种代为由1增至n时，对应算法所占用的空间也以某种单位由G(1)增至G(n)，这是我们称该算法的空间代价是G(n)

空间单位：一个简单变量（如实型、整型）所占存储空间的大小

3.大O表示法

- 1) 定义：当问题的规模逐渐增大，时间开销T(n)的增长趋势

“大O”表示法 $\lim_{n \rightarrow \infty} rate T(n) = O(F(n))$

如果存在正的常数c和 n_0 ，

当问题的规模 $n \geq n_0$ 后，

该算法的时间(或空间)代价 $T(n) \leq c \cdot F(n)$

则

称该算法的时间代价(或空间代价)为 $O(F(n))$ ，

这时

也称该算法的时间(或空间)代价的增长率为F(n)。

常用的时间复杂度按数量级递增排列：

- 常数阶 $O(1)$ 、
- 对数阶 $O(\log_2 n)$ 、
- 线性阶 $O(n)$ 、
- 线性对数阶 $O(n \log_2 n)$ 、
- 平方阶 $O(n^2)$ 、
- 立方阶 $O(n^3)$ 、
- k 次方阶 $O(n^k)$ 、
- 指数阶 $O(2^n)$ 。

2) 常用准则

加法准则：

$$T(n) = T_1(n) + T_2(n) \\ = O(f_1(n)) + O(f_2(n)) = O(f_1(n) + f_2(n))$$

乘法准则

$$T(n) = T_1(n) \times T_2(n) \\ = O(f_1(n)) \times O(f_2(n)) = O(f_1(n) \times f_2(n))$$

3) 举例

考虑如下程序：

```
{
    action1(...);
    action2(...);
}
```

其中action1的时间代价是 $T_1(n) = O(n^2)$ ，

action2的时间代价是 $T_2(n) = O(n^3)$ 。

按照加法规则，计算整个程序的时间代价为：

$$T(n) = T_1(n) + T_2(n) = O(\max(n^2, n^3)) = O(n^3)$$

```
(1) i=1; k=0;
while(i<n)
{ k=k+10*i; i++; }
```

◆ $T(n)=n-1$
 $\therefore T(n)=O(n)$
 ◇ 这个函数是按线性阶递增的

```
(3) i=1; j=0;
while(i+j<=n)
{ j++; i++; }
```

◆ $T(n)=n/2$
 $\therefore T(n)=O(n)$
 ◇ 虽然时间函数是 $n/2$ ，但其数量级仍是按线性阶递增的。

```
(2) x=91; y=100;
while(y>0)
If (x>100)
{ x=x-10; y--; }
else x++;
```

◆ $T(n)=O(1)$
 ◇ 这个程序看起来有点吓人，while循环总共运行了1100次，但是，我们看到这段程序的运行是和 n 无关的，就算它再循环一万年，我们也不

```
(4) x=n; // n>1
while (x>=2^(y+1))
y++;
```

◆ $T(n)=\log_2 n$
 $\therefore T(n)=O(\log_2 n)$
 ◇ 最坏的情况是 $y=0$ ，那么循环的次数是 $\log_2 x$ 次，也就是 $\log_2 n$ 次($n>1$)。这是一个按对数阶递增的函数。

2.常见的检索算法

1.基本的函数参数

int SeqSearch(int* element, int n, int key, int *pos)

数组名、数组长度、查找内容、是否找到/具体位置

2.顺序检索

```
int SeqSearch( int* element, int n, int key, int *pos)
/*顺序检索值为key的元素*/
{ int i;
  for ( i = 0; i < n; i++)
    if ( element[i] == key) { /*成功*/
      *pos = i;
      return 1;
    }
  *pos = i;
  return 0; /*失败*/
}
```

3.二分法检索（要求所有元素已经排序）

1) 其实质是逐步缩小查找区间的查找方法

①分别用low和high表示当前查找区间的上界和下界

②从 $\text{mid}=(\text{low}+\text{high})/2$ 的**中间位置**上开始查找

③若中间位置的元素 $\text{key}'==\text{key}$ ，那么检索成功

$\text{key}'>\text{key}$ ，在前半部分中继续进行二分法检索

$\text{key}'<\text{key}$ ，在后半部分中继续进行二分法检索

2) 代码

```
int binarySearch(int * element, int n, int key, int *position)
{
    int low=0, mid, high=n-1;
    while(low<=high)
    {
        mid=(low+high)/2;          /* 当前检索的中间位置 */
        if (element[mid]==key)    /* 检索成功 */
        {
            *position=mid;
            return(1);
        }
        else if (element[mid]>key) high=mid-1; /* 在左半区 */
        else low=mid+1;           /* 在右半区 */
    }
    *position=low;
    return(0); /* 检索失败 */
}
```

3.常见的排序算法

1.排序的基本概念

1) 正序：待排序列正好符合排序要求

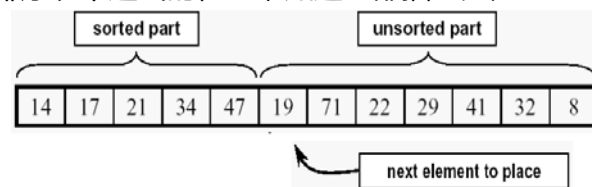
2) 逆序：待排序列逆转过来，正好符合排序要求

3) 排序的稳定性：在待排序的文件中，若存在多个排序码相同的记录，经过排序后记录的相对次序保持不变（跟初始数据中的次序一样），则这种排序方法称为是“稳定的”，否则是不稳定的。

2.插入排序

1) 直接插入排序（时间效率是 n^2 ）

①基本思想：模仿抓扑克牌的过程。逐个处理待排序的记录，每步将一个待排序的元素 R_i 按其排序码 k_i 插入到前面已排序表中适当的位置，知道全部插入完为止



②代码

```

void insertSort(int * record, int n) {
    /* 按递增序进行直接插入排序*/
    int i, j;
    int temp;
    for ( i = 1; i < n; i++ ) {
        temp = record[i];
        j = i-1;
        while ((temp < record[j]) && (j>=0) ) {
            record[j+1] = record[j];
            j--;
        }
        if( j!=(i-1) ) record[j+1] = temp;
    }
}

```

2) 二分法插入排序

3) 表插入排序

4) Shell排序

3.选择排序

1) 直接选择排序 (时间效率是 n^2)

①基本思想：在所有剩下的记录中选出排序码最小的记录，与剩下的第一个记录交换

②代码

```

void selectSort( int * record, int n )
{ // 直接选择排序，求递增序列
  int i, j, k; int temp;
  for( i = 0; i < n-1; i++ ) /* 做n-1趟选择排序*/
  { k=i;
    for(j=k+1; j<n; j++) /*找排序码最小的记录Rk */
      if(record[j]<record[k])
        k=j;
    if(k!=i) { /*记录Rk 与Ri 互换*/
      temp=record[i];
      record [i]= record [k];
      record [k]=temp;
    }
  }
}

```

2) 堆排序

4.交换排序

1) 冒泡排序 (时间效率是 n^2)

①基本思想：模仿体育老师编队形，前面的大值不停的与后面的进行比较，如果大于后面就换位，直到遇到比自己更大的就停止

②代码：

```

void bubbleSort( int * record, int n ) {
  int i, j, noswap;
  int temp;
  for(i=0; i<n-1; i++) /* 做n-1次起泡 */
  { noswap=1; /*置交换标志*/
    for(j=0; j<n-i-1; j++)
      if(record[j+1]<record[j]) {
        temp=record[j];
        record[j]=record[j+1];
        record[j+1]=temp;
        noswap=0; /*置交换标志*/
      }
    if(noswap) break; /*未发生记录交换，结束*/
  }
}

```

2) 快速排序

①基本思想：在待排的 n 个记录 $\{R_0, R_1 \dots R_{n-1}\}$ 中任选一个记录为标准 (通常为 R_0)，把这 n 个记录按“标准”分成前后两个部分：

把所有小于该标准的记录移到其左边；
把所有大于该记录的记录移到其右边
再对两个子序列分别重复上述过程，反复直到所有记录排好序
②代码：

```
void quickSort(int * record, int l, int r)
{ int i, j; int temp;
  if(l>=r) return; /*只有一个记录或无记录，则无需排序*/
  i=l; j=r; temp=record[i];
  while(i!=j) /*寻找R1的最终位置*/
  {
    while( (record[j]>=temp) && (j>i) )
      j--; /*从右到左扫描，查找第1个排序码小于temp.key的记录*/
    if(i<j)
      record[i++] = record[j];
    while( (record[i]<=temp) && (j>i) )
      i++; /*从左到右扫描，查找第1个排序码大于temp.key的记录*/
    if(i<j)
      record[j--] = record[i];
  }
  record[i] = temp; /*找到R1的最终位置*/
  quickSort(record, l, i-1); /*找到递归处理左区间*/
  quickSort(record, i+1, r); /*找到递归处理右区间*/
}
```

5.分配排序

1) 基数排序

6.归并排序

1) 二路归并排序

①基本思想：设文件中有n个记录，可以看成n个子文件，每个子文件中只包含一个记录。先将每两个子文件归并，得到n/2个部分排序的较大的子文件，每个子文件包含2个记录。再将子文件归并，如此反复，直到得到一个文件

②代码：不做要求

7.各种排序算法的理论时间代价

算法	最大时间	平均时间	最小时间	辅助空间 代价	稳定性
直接插入排序	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$	$\Theta(1)$	稳定
直接选择排序	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	不稳定
冒泡排序	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$	$\Theta(1)$	稳定
快速排序	$\Theta(n^2)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(\log n)$	不稳定
归并排序	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	稳定

4.课堂练习

第十四讲：递归与程序进阶

2017年12月14日 15:02

一、递推

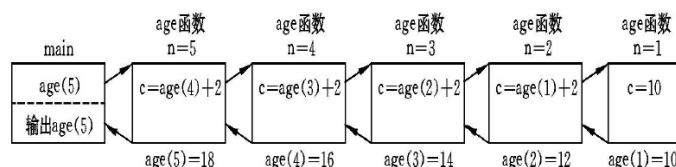
一、基本原理

1.函数不能嵌套定义，但可以嵌套调用，特别地，可以（通过参数的变化）自己调用自己

2.例题1：

- 有5个人坐在一起，问第5个人多少岁？
 - 第5个人说比第4个人大2岁。
 - 第4个人说比第3个人大2岁。
 - 第3个人说比第2个人大2岁。
 - 第2个人说比第1个人大2岁。
 - 第1个人他说是10岁

如果我们用一个函数（age）来求出第五个人的年龄，用图示来表示就是下面这样

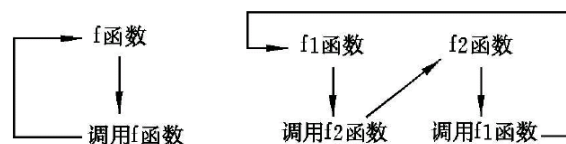


写成程序就是

```
#include<iostream.h>
int age(int n) {
    int c;
    if(n == 1) c = 10;
    else c = age(n-1)+2;
    return(c);
}
void main() {
    printf(“第五个人的年龄是: %d”, age(5) );
}
```

2.递归的定义

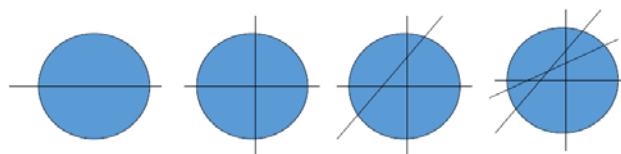
1.递归：在调用一个函数的过程中又出现直接或间接地调用该函数本身。例如下面两个图例



程序中不应该出现无终止的递归调用，必须控制**只有在某一条件成立时**才继续执行递归调用，否则就不再继续，称为“递归出口”

2.例题2：

- 切饼，100刀最多能切多少块？



通过归纳法我们可以从数学上知道，这个结果满足 $q(n)=q(n-1)+n$

```

#include <stdio.h>
int q(int n) {
    if (n == 0) return 1;
    else return( n + q(n-1));
}

int main() {
    printf("%4d", q(100) );
    return 0;
}

```

在这个程序里面， $q(0)=1$ 就是递归出口

3.递归的应用

1.递推数列

例如我们要来算一个阶乘数列，有

$\text{fact}(n) = n * \text{fact}(n-1)$ (通项公式) ;
 $\text{fact}(1) = 1$ (边界条件)

如果用递归的方式来得到阶乘的结果，那么程序应该这样写

```

#include <stdio.h>
int fact(int n)
{
    if (n == 1) return 1;
    else return( n * fact(n-1));
}

int main( )
{
    printf("&d\n", fact(10));
    return 0;
}

```

要特别注意，“递推”与“递归”有着根本的区别。递推中，后续的运算依赖于一致的条件，当前的运算是下一步运算的基础，运算的顺序是“从前往后”；而递归中，出发点不在初始条件上，而放在求解的目标上，它是从所求的位置向出发逐步调用本身的求解过程，直到递归的边界（即初始条件），运算的顺序是“从后往前”

2.求斐波那契数列的第n个数

$F(0) = 0;$
 $F(1) = 1;$
 $F(n) = F(n-1) + F(n-2);$

边界条件
通项公式

函数体按照递归的写法就是：

```

int F(int n) {
    if(n==0) return 0;
    if(n==1) return 1;
    return ( F(n-1)+F(n-2));
}

```

3.字符串反向

1) 目标：编写程序接受一个字符串的输入，将字符串反向输出

2) 程序

```

#include<stdio.h>
void echo() {
    char c1;
    scanf("%c",&c1);
    if (c1!= '\n') echo();
    printf("%c", c1);
}
void main() {
    echo();
}

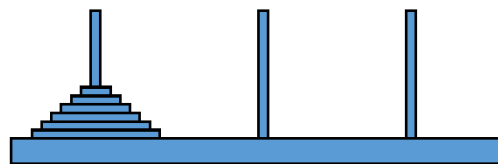
```

解释：如果输入的字符不是回车，那么就返回去接着执行echo，也就是读入字符，而并不执行printf。直到读入回车后一层一层执行printf，将输入的内容倒着输出

4.汉诺塔问题

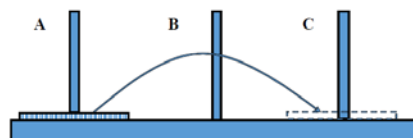
相传在古代印度的Bramah庙中，有位僧人整天把三根柱子上的金盘倒来倒去，原来他是想把64个一个比一个小的金盘从一根柱子上移到另一根柱子上去。移动过程中恪守下述规则：每次只允许移动一只盘，且大盘不得落在小盘上面。

有人会觉得这很简单，真的动手移盘就会发现，如以每秒移动一只盘子的话，按照上述规则将64只盘子从一个柱子移至另一个柱子上所需时间约为5800亿年。



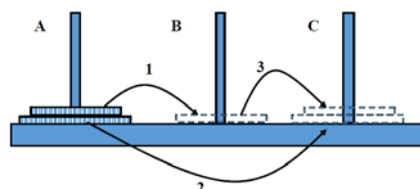
先理出这个问题的递推思路

- 1、在A柱上只有一只盘子，假定盘号为1，这时只需将该盘从A搬至C，一次完成，记为move 1 from A to C

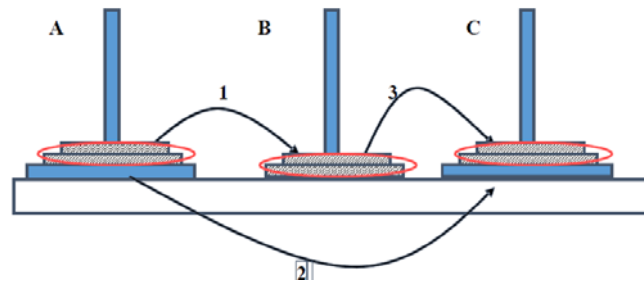


- 2、在A柱上有二只盘子，1为小盘，2为大盘。
 第（1）步将1号盘从A移至B；
 第（2）步将2号盘从A移至C；
 第（3）步再将1号盘从B移至C；
 这三步记为：

move 1 from A to B;
 move 2 from A to C;
 move 3 from B to C;



- 3、在A柱上有3只盘子，从小到大分别为1号，2号，3号
 第（1）步将1号盘和2号盘视为一个整体；先将二者作为整体从A移至B。这一步记为
`move( 2, A, C, B)`
 第（2）步将3号盘从A移至C，一次到位。记为
`move 3 from A to C`
 第（3）步处于B上的作为一个整体的2只盘子，再移至C。这一步记为
`move( 2, B, A, C)`



看完第三步，我们的思路就出来了。将上面的 $n-1$ 个盘子视为一个整体，`move(n,A,B,C)`就分解成为了三步：`move(n-1,A,C,B)`（也就是将上面的 $n-1$ 只盘子作为一个整体从A经过C移动到B），`n: A to C`（将 n 号盘从A移动到C），`move(n-1,B,A,C)`（将上面的 $n-1$ 只盘子作为一个整体从B经过A移动到C）

写出程序就是

```
#include<stdio.h>
void move(int m, char A, char B, char C)
//表示将m个盘子从A经过B移动到C
{ if (m==1) //如果m为1,则为直接可解结点
    printf("move 1# from %c to %c\n", A, C);
    else
    {
        move(m-1,A,C,B); //递归调用move(m-1)
        printf("move 1# from %c to %c\n", A, C); //直接可解结点
        move(m-1,B,A,C); //递归调用move(m-1)
    }
}

int main()
{
    int n; //整型变量, n为盘数
    printf("请输入盘数n=\n");
    scanf("%d",&n); //输入盘子数目正整数n
    printf("在3根柱子上移 %d 只盘的步骤为:\n");
    move(n,'a','b','c'); //调用函数
    move(n,'a','b','c')
    return 0;
}
```

二、编程进阶

1.多交叉路口交通信号灯的管理问题

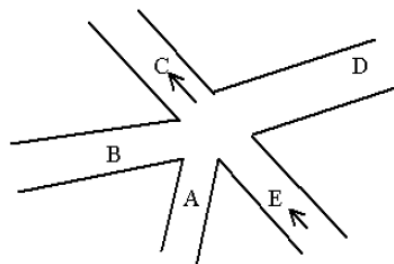


图 1.1 一个交叉路口的模型

其中，E和C按照箭头的方向设定为单行道
将不能同时进行的方向用线连接起来，得到图式模型

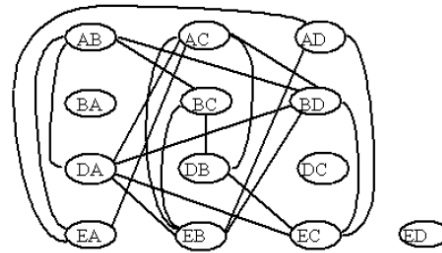


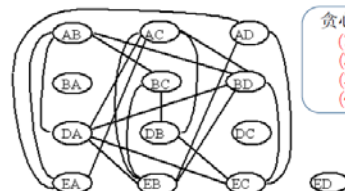
图 1.2 交叉路口的图式模型

为了叙述方便，我们下面把A→B简写成AB，并且用一个小椭圆把它框起来，在不能同时行驶的路线间画一条连线(表示冲突)，便可以得到图1.2。

问题：要求将图1.2中的结点分组，使有线相连(互相冲突)的结点不在同一个组里。

(二) 算法设计 (贪心法)

- 先用一种颜色给尽可能多的结点上色；
- 然后用另一种颜色在未着色结点中给尽可能多的结点上色；
- 如此反复直到所有结点都着色为止。



贪心法的一个解：
(1) 红色：AB AC AD BA DC ED
(2) 蓝色：BC BD EA
(3) 绿色：DA DB
(4) 白色：EB EC

图 1.2 交叉路口的图式模型

利用着色法，可以把这些节点按照独立关系分组

第十五讲：总结&哈弗曼编码

2017年12月21日 15:14

一、问题的提出

数据通信的二进制编码问题描述：

设需要编码的字符集合 $d = \{d_1, d_2, \dots, d_n\}$, $w = \{w_1, w_2, \dots, w_n\}$ 为 d 中各种字符出现的频率，要对 d 里的字符进行二进制编码，使得

- ① 通信编码总长最短
- ② 若 $d_i \neq d_j$, 则 d_i 的编码不可能是 d_j 的编码的前缀。（这样就使得译码可以一个字符一个字符地进行，不需要在字符与字符之间添加分隔符）

例如，你不能把 a 编码成 00 ，然后把 b 编码成 001

2. 解决方案1

1) 定长编码：所有字符都具有相同的编码长度

如：26个字符 $A-Z$ 编码成 $00000-11001$

2) 优点：编码简单

3) 缺点：各个字符出现概率不相等时，文章的编码长度不是最短的

3. 不定长编码

不定长编码：假定字符出现频率分布如下 $\{p_1, p_2, \dots, p_m\}$ ，根据减少信息串编码长度的要求，频率最大的字符其编码位数应该最小。

- 假定各个字符得到的编码位数为 $\{l_1, l_2, \dots, l_m\}$ ，则总的信息串的长度为：

$$AL = \sum_{i=1}^m P_i L_i$$

1) AL 要最小，定性地说就是 p_i 越大，则 l_i 越小

2) 问题核心：最经常出现的字符编码最短

避免出现二义性问题

3) 哈夫曼树：哈夫曼树是一棵二叉树

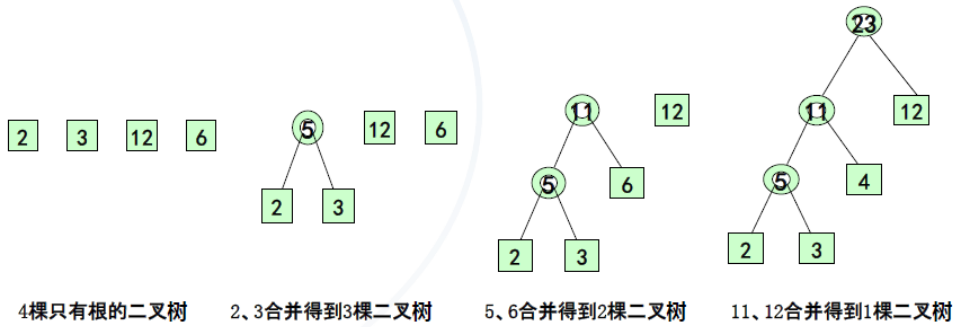
利用根节点到叶节点的路径进行编码

所有的字符都在叶节点上，因此不会出现二义性

二叉树中，带权路径长度最小

4) 哈弗曼树的构造过程

- 设字符集 {A, B, C, D} 出现的频率（对应的权值）分别为 {2, 3, 4, 11}

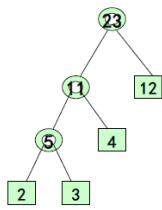


左右选择不同，得到形态不同的Huffman树，
但它们的**带权路径长度**相同。

从最小的权值开始进行从底层向上的构造

5) 用数组存储哈夫曼树

设待编码的字符个数为 m ，则增加结点个数为 $m-1$ ，因此最后得到的Huffman树必定有 $2m-1$ 个结点。因此，用 $2m-1$ 个元素的一维数组就可以存储该Huffman树。



- 边数=内部节点个数*2
- 边数=外部节点数+内部节点数-1

结点作为一个结构体，包含权重、父节点在数组中的存储位置（根的父亲节点设为-1），左子节点位置，右子节点位置（外部结点的两侧结点位置设为-1）

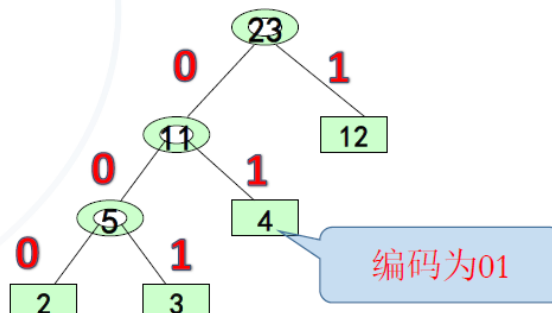
- **ww**: 以该结点为根的子树中所有外部结点的加权和。
- **parent**: 父结点在数组中的存储位置（下标），根无父，则可以设为-1。
- **llink**: 左子节点位置；对于外部结点，设为-1。
- **rlink**: 右子节点位置；对于外部结点，设为-1。

6) 利用哈夫曼树进行编码

根据字符出现的概率，建立Huffman树以后，

- 把从每个结点引向其左子女的边标上**号码 0**
- 把从每个结点引向其右子女的边标上**号码 1**

从根到每个叶子的路径上的号码连接起来就是这个叶子代表的字符的编码。



7) 哈夫曼编码的译码

最终的译码过程原则就是：读到能译出来东西的程度就立马翻译，而不用考虑二义性的问题（因为在设计上已经排除了）

上机实验常见问题分析

2017年11月15日 19:18

上机实验常见问题分析

一、代码格式

良好的代码格式

- 1) 缩进：缩进的单位一般是一个tab
大括号内的语句要缩进
for/if/while/switch等分支语句块内的语句要缩进
- 2) 换行：每条语句都要换行
过长的表达式应该换行写
- 3) 空行：按照“意群”分割你的代码，中间增加一些空行以增加代码的可读性
- 4) 空格：运算符前后一般空一格
- 5) 括号：可能引起错误的优先级运算中，尽量多加括号使得代码对你和对别人来说都更容易读懂和找到错误所在
- 6) 其他：尽量多写注释
尽量使用有意义的变量名

二、语法问题

常见的语法问题

- 1) “=” 和 “==” 的混用
if(a=1)下面的语句一定执行，因为表达式a=1的返回值是1
if(a=0)下面的语句一定执行，因为表达式a=0的返回值是0
- 2) 整数相除时的精度损失
- 3) scanf的输入变量忘记“&”：如果忘了加&符号，会引起内存访问的错误
反之，如果在printf里面加了&，那么输出的结果是地址
- 4) scanf输入格式和变量类型不对应
- 5) for/if语句后的一组分支语句块没有用大括号括起来
- 6) 符号错误：标点符号：标点符号都是英文的不是中文的
每一条语句都要加分号
圆括号、大括号都要匹配
相似字符：小写字母l和数字1
字母o和数字0
尽量避免使用这些符号，使用有意义的变量名

三、逻辑问题

1.数据初始化问题

- 1) 使用变量前没有初始化：常见于求和、标志位等变量，忘了初始化数组也很常见

2.数组相关的问题

- 1) 数组开辟的大小：数组的尺寸必须用常量（字面量）来定义

一般开辟一个超过最大值的数组，然后只用其中一部分就可以了，避免在寻址的时候越界

2) 数组、指针的访问越界

3. 输入问题

1) 多组输入：确定case数量的输入

不确定case数量的输入：根据输入值判断是否结束：`while(scanf("%lf, %c") != 'EOF')`

根据文件尾判断是否结束

四、程序调试

逐条执行 next line

进入函数体内部 into function

跳出循环 skip function